



UNIVERSIDAD TECNOLÓGICA DE LA
HUASTECA HIDALGUENSE

Organismo Descentralizado de la Administración Pública Estatal

**INGENIERIA EN TECNOLOGÍAS DE LA INFORMACIÓN ÁREA DE
DESARROLLO DE SOFTWARE**



ASIGNATURA: EXTRACCIÓN DE CONOCIMIENTOS EN BASES DE DATOS

ACTIVIDAD: PROYECTO DE MES 1

NOMBRE DEL ALUMNO:

PABLO RUIZ SANTOS –20230031

DOCENTE: DR. EFRÉN JUÁREZ CASTILLO

CUATRIMESTRE: 9º

GRUPO “C”

DATASET: ONLINE SHOPPERS PURCHASING INTENTION DATASET

URL: <https://www.kaggle.com/datasets/imakash3011/online-shoppers-purchasing-intention-dataset>

I. Comprensión del negocio

Introducción

El comercio electrónico se ha convertido en una actividad importante para muchas empresas, ya que permite ofrecer productos y servicios a través de plataformas digitales. Sin embargo, uno de los principales retos de una tienda en línea es que no todas las visitas realizadas por los usuarios terminan en una compra. Muchas personas ingresan al sitio, revisan productos, comparan información o abandonan la página sin generar ingresos para la empresa.

Para analizar este problema se utiliza el dataset **Online Shoppers Purchasing Intention Dataset**, el cual contiene información sobre sesiones de usuarios dentro de una tienda en línea. De acuerdo con Sakar y Kastro (2018), este conjunto de datos está formado por 12,330 sesiones, donde cada sesión pertenece a un usuario diferente durante un periodo de un año. Esto permite reducir sesgos relacionados con campañas específicas, fechas especiales o comportamientos repetidos de los mismos usuarios.

El tema general del dataset es la **intención de compra de los usuarios en una tienda en línea**. Sus atributos permiten analizar aspectos como el número de páginas visitadas, el tiempo que el usuario permanece en distintos tipos de páginas, la tasa de rebote, la tasa de salida, el valor de las páginas, el tipo de visitante, el mes de la sesión, el sistema operativo, el navegador, la región, el tipo de tráfico y si la visita ocurrió durante fin de semana. La variable principal del análisis es Revenue, la cual indica si una sesión generó ingresos o no; por esta razón, el dataset puede utilizarse para un problema de clasificación binaria (Sakar & Kastro, 2018).

Analizar este dataset puede ser útil porque permite identificar patrones de comportamiento relacionados con la posibilidad de compra. Por ejemplo, una tienda en línea podría usar este tipo de análisis para conocer qué características tienen las sesiones que sí

generan ingresos y cuáles son más comunes en las sesiones que no terminan en compra. Esta información puede apoyar decisiones relacionadas con estrategias de marketing, promociones personalizadas, segmentación de clientes, mejora de la experiencia del usuario y optimización del sitio web.

Objetivo de la clasificación

El objetivo de la clasificación en este proyecto es predecir la variable **Revenue**, la cual indica si una sesión de usuario dentro de una tienda en línea generó ingresos o no. Esta variable es de tipo binario, ya que puede tomar dos posibles valores: `True`, cuando la sesión terminó en una compra o generó ingresos, y `False`, cuando la sesión no generó ingresos. De acuerdo con Sakar y Kastro (2018), el dataset **Online Shoppers Purchasing Intention Dataset** utiliza la variable `Revenue` como etiqueta de clase, por lo que es adecuado para desarrollar un modelo de clasificación.

La utilidad de realizar esta clasificación consiste en identificar patrones de comportamiento que permitan estimar la intención de compra de los usuarios. A partir de variables como el número de páginas visitadas, el tiempo de navegación, la tasa de rebote, la tasa de salida, el valor de las páginas, el tipo de visitante y otros datos de la sesión, el modelo puede aprender a diferenciar entre sesiones que probablemente generen ingresos y sesiones que no.

Esta clasificación puede ser útil para una tienda en línea porque permitiría apoyar la toma de decisiones relacionadas con estrategias de marketing, promociones personalizadas, recomendaciones de productos y mejora de la experiencia del usuario. Por ejemplo, si el modelo detecta que una sesión tiene alta probabilidad de generar ingresos, la empresa podría mostrar ofertas o productos relacionados para aumentar la posibilidad de compra. En cambio, si se

detectan sesiones con baja probabilidad de generar ingresos, se podrían analizar las causas del abandono y mejorar aspectos del sitio web.

Posible aplicación del modelo.

Una posible aplicación real del modelo sería en una tienda en línea que busca mejorar sus estrategias de venta y aumentar la cantidad de sesiones que terminan en compra. El modelo de clasificación podría analizar el comportamiento de los usuarios durante su navegación y estimar si una sesión tiene alta o baja probabilidad de generar ingresos, tomando como referencia la variable Revenue del dataset Online Shoppers Purchasing Intention Dataset.

En una situación real, este modelo podría utilizarse dentro de un sistema de comercio electrónico para apoyar decisiones relacionadas con marketing digital, promociones personalizadas, recomendaciones de productos y mejora de la experiencia del usuario. Por ejemplo, si el modelo detecta que un usuario tiene alta probabilidad de comprar, la tienda podría mostrarle productos relacionados, descuentos especiales o recordatorios para finalizar su compra. En cambio, si el modelo identifica que una sesión tiene baja probabilidad de generar ingresos, la empresa podría analizar qué factores influyen en el abandono, como una alta tasa de rebote, poco tiempo de navegación o baja interacción con páginas de productos.

Los resultados del modelo también podrían apoyar decisiones estratégicas, como identificar qué tipo de visitantes tienen mayor intención de compra, qué meses presentan mejor comportamiento comercial, qué fuentes de tráfico generan más ingresos o qué características de navegación se relacionan con una compra. Con esta información, la empresa podría optimizar sus campañas publicitarias, mejorar la estructura del sitio web, diseñar promociones más efectivas y enfocar sus recursos en los usuarios con mayor posibilidad de conversión.

Por lo tanto, la aplicación del modelo no solo consistiría en predecir si una sesión generará ingresos, sino también en utilizar esa predicción como apoyo para tomar decisiones que mejoren el rendimiento de una tienda en línea. Esto puede ayudar a reducir el abandono de usuarios, aumentar las ventas y mejorar la experiencia general del cliente dentro de la plataforma.

II. Comprensión de los datos

En esta etapa se analiza el dataset antes de realizar transformaciones importantes. El objetivo es conocer su estructura, el tipo de información que contiene, la variable que se desea predecir, la cantidad de registros, los atributos disponibles y las características generales de los datos. Esta revisión es importante porque permite detectar posibles problemas, como valores faltantes, variables categóricas, distribuciones sesgadas, valores atípicos o relaciones relevantes entre atributos.

- Nombre del dataset: Online Shoppers Purchasing Intention Dataset (Intención de compra de los compradores en línea)
- URL: <https://www.kaggle.com/datasets/imakash3011/online-shoppers-purchasing-intention-dataset>
- Numero de registros: 12330
- Numero de atributos: 18

Figura 2.1. Dimensiones del dataset Online Shoppers Purchasing Intention.

```
[13]: df.shape  
[13]: (12330, 18)
```

Nota. La salida de `df.shape` muestra que el dataset contiene **12,330 registros** y **18 atributos**.

En este proyecto, la variable objetivo es Revenue, mientras que las demás columnas se consideran variables predictoras. El propósito del análisis es identificar patrones en el comportamiento de navegación de los usuarios que puedan relacionarse con la generación de ingresos dentro de la tienda en línea.

El dataset contiene **18 columnas en total**. De estas, **17 corresponden a variables independientes** y **1 corresponde a la variable dependiente**, llamada Revenue. La variable Revenue tiene dos posibles valores: True, cuando la sesión generó ingresos, y False, cuando la sesión no generó ingresos.

Descripción de los atributos

El dataset **Online Shoppers Purchasing Intention Dataset** contiene 18 atributos originales relacionados con sesiones de usuarios dentro de una tienda en línea. Estos atributos permiten analizar el comportamiento de navegación de los visitantes, las características técnicas de la sesión, el contexto temporal y el resultado final relacionado con la generación de ingresos.

Para esta etapa se revisaron todos los atributos originales del dataset con el propósito de identificar su significado, su tipo de atributo, el tipo de dato registrado en Python y la presencia o ausencia de valores nulos. Esta revisión es importante porque permite conocer la estructura de los datos

antes de aplicar transformaciones, limpieza o técnicas de modelado.

De acuerdo con Sakar y Kastro (2018), el dataset contiene variables numéricas y categóricas relacionadas con sesiones de usuarios en comercio electrónico, y la variable revenue se utiliza como etiqueta de clase para el problema de clasificación.

Figura 2.2. Tipos de datos de los atributos del dataset.

```
[8]: #Comprobar los tipos de datos

[9]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 12330 entries, 0 to 12329
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Administrative                        12330 non-null  int64
1   Administrative_Duration              12330 non-null  float64
2   Informational                        12330 non-null  int64
3   Informational_Duration              12330 non-null  float64
4   ProductRelated                      12330 non-null  int64
5   ProductRelated_Duration            12330 non-null  float64
6   BounceRates                         12330 non-null  float64
7   ExitRates                          12330 non-null  float64
8   PageValues                          12330 non-null  float64
9   SpecialDay                          12330 non-null  float64
10  Month                              12330 non-null  str
11  OperatingSystems                   12330 non-null  int64
12  Browser                           12330 non-null  int64
13  Region                            12330 non-null  int64
14  TrafficType                       12330 non-null  int64
15  VisitorType                       12330 non-null  str
16  Weekend                           12330 non-null  bool
17  Revenue                           12330 non-null  bool
dtypes: bool(2), float64(7), int64(7), str(2)
memory usage: 1.5 MB
```

Figura 2.3. Revisión de valores nulos por atributo.

```
[11]: df.isnull().sum()

[11]: Administrative      0
      Administrative_Duration  0
      Informational      0
      Informational_Duration  0
      ProductRelated      0
      ProductRelated_Duration  0
      BounceRates         0
      ExitRates           0
      PageValues          0
      SpecialDay          0
      Month              0
      OperatingSystems    0
      Browser            0
      Region             0
      TrafficType        0
      VisitorType        0
      Weekend            0
      Revenue            0
      dtype: int64

[ ]:
```

Tabla 1. Descripción técnica de los atributos del dataset.

Atributo	Significado o descripción	Tipo de atributo	Tipo de dato en Python	Valores nulos
Administrative	Número de páginas administrativas visitadas durante la sesión.	Razón	int64	Ausencia de valores nulos
Administrative_Duration	Tiempo total que el usuario permaneció en páginas administrativas.	Razón	float64	Ausencia de valores nulos
Informational	Número de páginas informativas visitadas durante la sesión.	Razón	int64	Ausencia de valores nulos
Informational_Duration	Tiempo total que el usuario permaneció en páginas informativas.	Razón	float64	Ausencia de valores nulos
ProductRelated	Número de páginas relacionadas con productos visitadas durante la sesión.	Razón	int64	Ausencia de valores nulos
ProductRelated_Duration	Tiempo total que el usuario permaneció en páginas relacionadas con productos.	Razón	float64	Ausencia de valores nulos
BounceRates	Tasa de rebote de la sesión; indica la proporción de usuarios que abandonan el sitio sin continuar navegando.	Razón	float64	Ausencia de valores nulos
ExitRates	Tasa de salida; representa la frecuencia con la que una página fue la última visitada en la sesión.	Razón	float64	Ausencia de valores nulos
PageValues	Valor promedio asignado a las páginas visitadas antes de una posible transacción.	Razón	float64	Ausencia de valores nulos
SpecialDay	Cercanía de la sesión a un día especial o fecha comercial importante.	Razón	float64	Ausencia de valores nulos
Month	Mes en que ocurrió la sesión de navegación.	Nominal	object	Ausencia de valores nulos
OperatingSystems	Sistema operativo utilizado por el visitante.	Nominal	int64	Ausencia de valores nulos

Atributo	Significado o descripción	Tipo de atributo	Tipo de dato en Python	Valores nulos
Browser	Navegador utilizado por el visitante.	Nominal	int64	Ausencia de valores nulos
Region	Región geográfica del visitante.	Nominal	int64	Ausencia de valores nulos
TrafficType	Tipo de tráfico o canal por el cual el usuario llegó al sitio web.	Nominal	int64	Ausencia de valores nulos
VisitorType	Tipo de visitante, por ejemplo visitante nuevo o recurrente.	Nominal	object	Ausencia de valores nulos
Weekend	Indica si la sesión ocurrió durante fin de semana.	Nominal	bool	Ausencia de valores nulos
Revenue	Indica si la sesión generó ingresos o no. Es la variable objetivo.	Nominal	bool	Ausencia de valores nulos

Como se observa en la tabla anterior, el dataset contiene variables de tipo numérico, categórico y booleano. Las variables numéricas, como ProductRelated, ProductRelated_Duration, BounceRates, ExitRates y PageValues, describen el comportamiento de navegación del usuario dentro del sitio web. Por otro lado, variables como Month, VisitorType, Weekend, OperatingSystems, Browser, Region y TrafficType representan características categóricas de la sesión.

La variable Revenue es la variable dependiente del proyecto, ya que indica si una sesión generó ingresos o no. Las demás variables serán consideradas como posibles variables independientes para entrenar los modelos de clasificación.

Después de revisar los valores nulos mediante `df.isnull().sum()`, se identificó que el dataset no presenta valores faltantes en sus columnas originales. Esto significa que, en esta etapa

inicial, no existen datos ausentes que puedan afectar directamente el análisis exploratorio. Sin embargo, en la etapa de preparación de datos se seguirá lo solicitado en la actividad, donde se indica que si el dataset no contiene valores faltantes, será necesario generar algunos valores faltantes aleatoriamente para realizar el ejercicio de tratamiento de nulos.

Análisis exploratorio de datos, EDA

El análisis exploratorio de datos, conocido como EDA, permite conocer el comportamiento general del dataset antes de realizar transformaciones o entrenar modelos de clasificación. En esta etapa se revisan las estadísticas descriptivas, la distribución de las variables, posibles valores atípicos, patrones entre atributos y relaciones que puedan existir con la variable objetivo Revenue.

Para este análisis se utilizaron estadísticas descriptivas como media, mediana, desviación estándar, valores mínimos, máximos y percentiles. También se realizaron visualizaciones como histogramas, boxplots, matriz de correlación y scatter plots entre atributos numéricos.

Tabla 2.4 Estadísticas descriptivas de las variables numéricas.

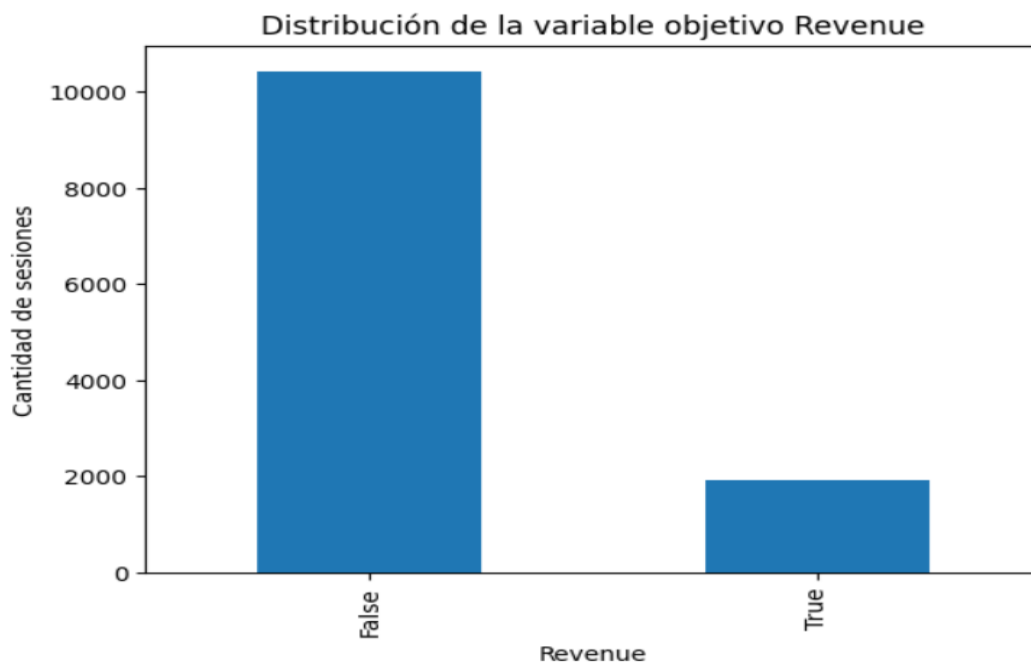
[12]:

	mean	mediana	std	min	25%	50%	75%	max
Administrative	2.315166	1.000000	3.321784	0.0	0.000000	1.000000	4.000000	27.000000
Administrative_Duration	80.818611	7.500000	176.779107	0.0	0.000000	7.500000	93.256250	3398.750000
Informational	0.503569	0.000000	1.270156	0.0	0.000000	0.000000	0.000000	24.000000
Informational_Duration	34.472398	0.000000	140.749294	0.0	0.000000	0.000000	0.000000	2549.375000
ProductRelated	31.731468	18.000000	44.475503	0.0	7.000000	18.000000	38.000000	705.000000
ProductRelated_Duration	1194.746220	598.936905	1913.669288	0.0	184.137500	598.936905	1464.157214	63973.522230
BounceRates	0.022191	0.003112	0.048488	0.0	0.000000	0.003112	0.016813	0.200000
ExitRates	0.043073	0.025156	0.048597	0.0	0.014286	0.025156	0.050000	0.200000
PageValues	5.889258	0.000000	18.568437	0.0	0.000000	0.000000	0.000000	361.763742
SpecialDay	0.061427	0.000000	0.198917	0.0	0.000000	0.000000	0.000000	1.000000
OperatingSystems	2.124006	2.000000	0.911325	1.0	2.000000	2.000000	3.000000	8.000000
Browser	2.357097	2.000000	1.717277	1.0	2.000000	2.000000	2.000000	13.000000
Region	3.147364	3.000000	2.401591	1.0	1.000000	3.000000	4.000000	9.000000
TrafficType	4.069586	2.000000	4.025169	1.0	2.000000	2.000000	4.000000	20.000000

En la tabla de estadísticas descriptivas se observan medidas como la media, mediana, desviación estándar, valores mínimos, máximos y percentiles de las variables numéricas. Estas medidas permiten conocer la escala de cada atributo y detectar posibles diferencias importantes entre variables.

Se puede observar que algunas variables presentan valores muy dispersos, especialmente aquellas relacionadas con la duración de navegación, como `ProductRelated_Duration`, `Administrative_Duration` e `Informational_Duration`. Esto indica que algunas sesiones tuvieron tiempos de navegación mucho más altos que otras. También se puede identificar que variables como `PageValues`, `BounceRates` y `ExitRates` pueden presentar distribuciones sesgadas, lo cual será importante considerar en etapas posteriores del proyecto.

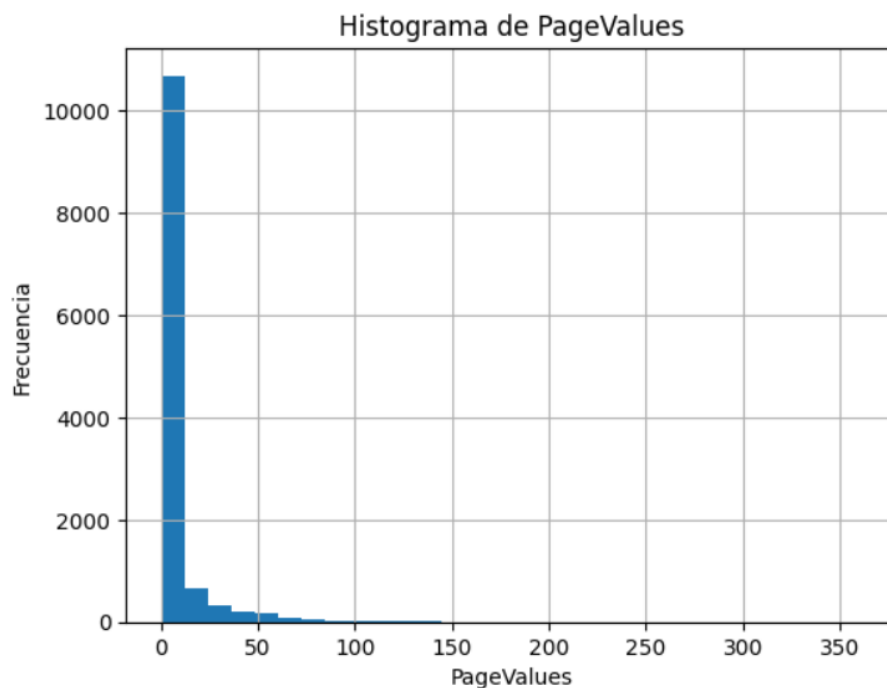
Figura 2.5 Distribución de la variable objetivo Revenue.



En la Figura 2.5 se observa la distribución de la variable objetivo Revenue. Esta variable indica si una sesión generó ingresos o no. La gráfica permite identificar que existe una mayor cantidad de sesiones con valor False, es decir, sesiones que no generaron ingresos, en comparación con las sesiones con valor True.

Este hallazgo muestra un posible desbalance entre clases, ya que las sesiones que no terminan en compra son más frecuentes. Esta situación es común en plataformas de comercio electrónico, debido a que muchos usuarios visitan el sitio para consultar productos, comparar precios o navegar sin realizar una compra. Para el modelado, este desbalance es importante porque puede influir en el desempeño de los algoritmos de clasificación.

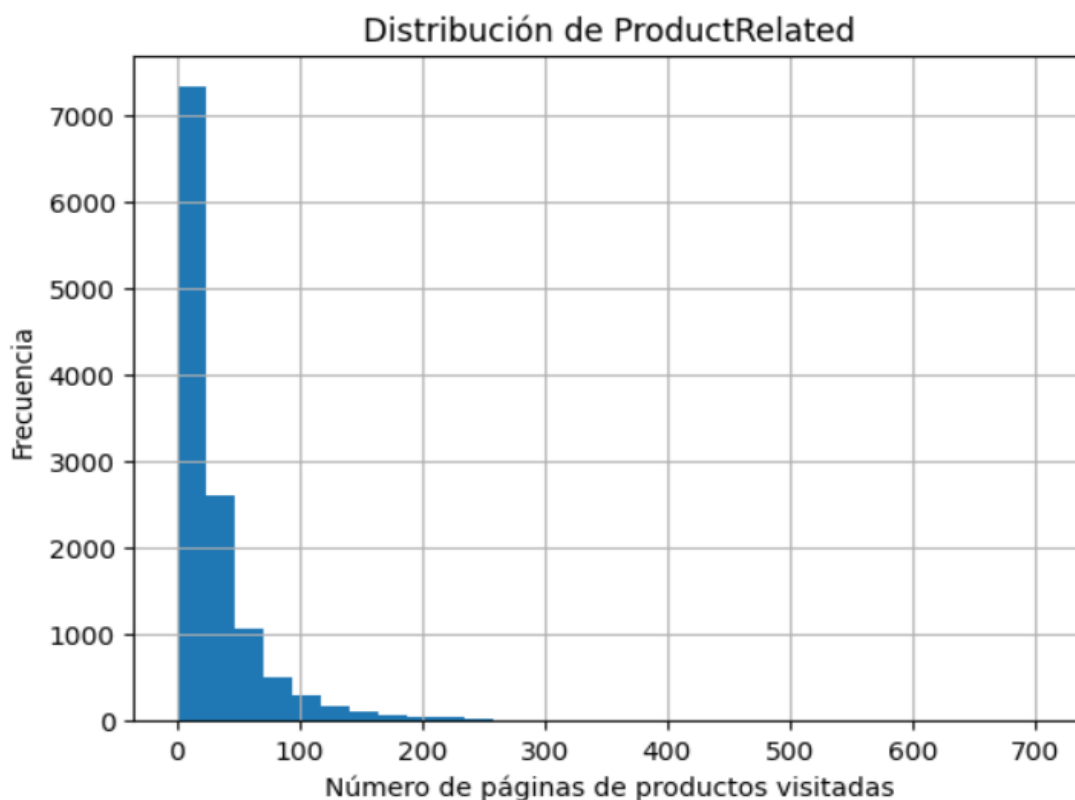
Figura 2.6 Histogramas de PageValues



En la Figura 2.6 se observa la distribución de la variable PageValues, la cual representa el valor promedio asignado a las páginas visitadas antes de una posible transacción. La gráfica permite notar que la mayoría de los valores se concentran cerca de cero, lo que indica que muchas sesiones tienen bajo valor para la tienda en línea.

También se observa una distribución sesgada hacia la derecha, debido a que existen pocas sesiones con valores altos. Este comportamiento puede ser relevante para el modelo, ya que las sesiones con valores altos en PageValues podrían estar más relacionadas con una mayor probabilidad de generar ingresos.

Figura 2.7 Histogramas de ProductRelated

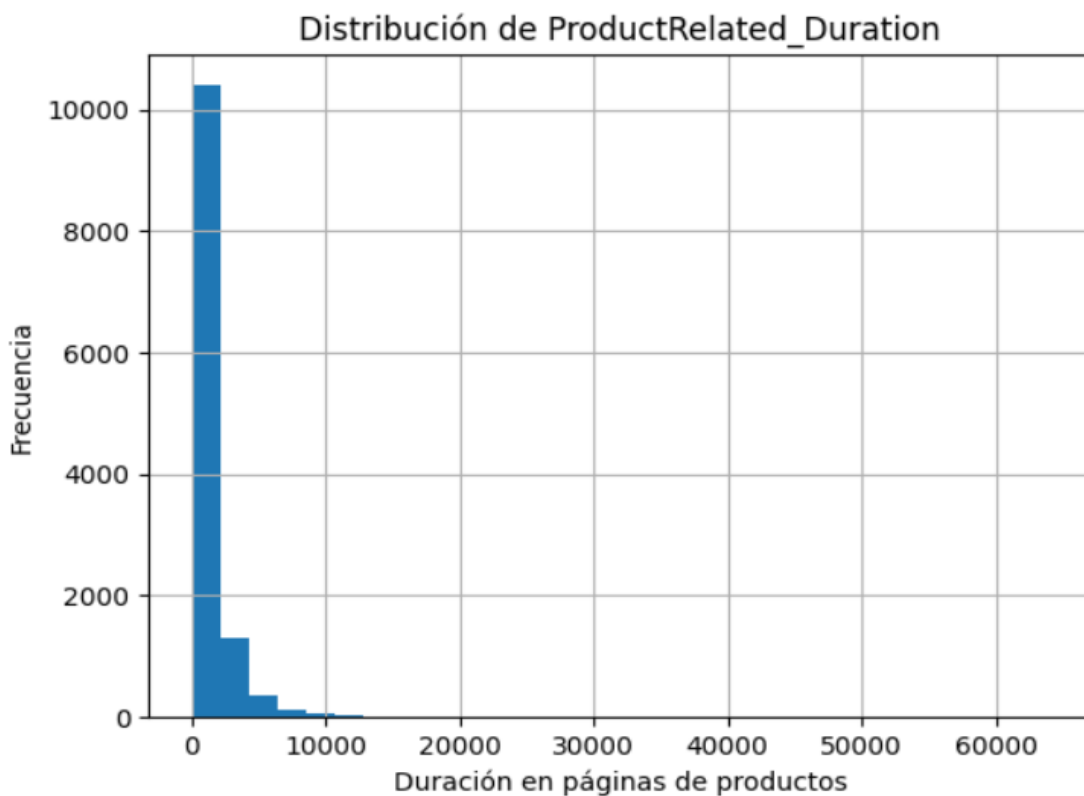


En la figura 2.7 se observa la distribución de la variable ProductRelated, la cual representa la cantidad de páginas relacionadas con productos que fueron visitadas durante una sesión. La mayoría de los registros se concentran en valores bajos o medios, lo que indica que

muchos usuarios visitan pocas páginas de productos antes de abandonar el sitio o finalizar su navegación.

También se puede observar una distribución sesgada hacia la derecha, ya que existen algunas sesiones donde los usuarios visitaron una cantidad mucho mayor de páginas de productos. Este comportamiento puede representar usuarios con mayor interés de compra, debido a que revisaron más productos dentro de la tienda en línea. Por esta razón, ProductRelated puede ser una variable importante para el modelo de clasificación, ya que la interacción con páginas de productos podría relacionarse con la variable objetivo Revenue.

Figura 2.8 Histogramas de ProductRelated_Duration

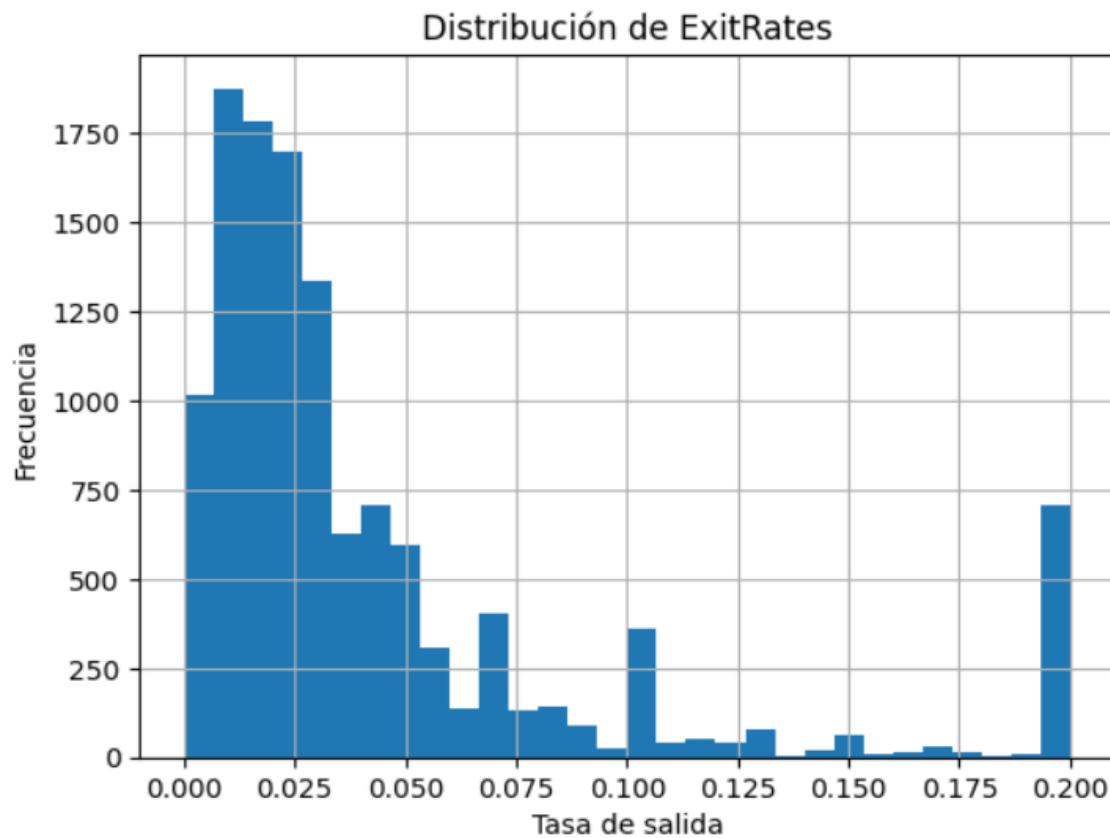


En la figura 2.8 se muestra la distribución de ProductRelated_Duration, que representa el tiempo total que los usuarios permanecieron en páginas relacionadas con productos durante una sesión. Es importante mencionar que la fuente del dataset no especifica si esta duración está medida en segundos, minutos u otra unidad exacta, por lo que en este análisis se interpreta como una unidad de tiempo registrada por la plataforma (Sakar & Kastro, 2018).

Se observa que una gran cantidad de sesiones tienen valores bajos de duración, mientras que pocas sesiones presentan duraciones muy altas. Este comportamiento indica una distribución sesgada hacia la derecha, donde la mayoría de los usuarios pasan poco tiempo revisando productos, pero algunos permanecen mucho más tiempo en esta sección.

Los valores altos pueden considerarse relevantes, ya que podrían representar sesiones con mayor interés en los productos. Sin embargo, también pueden aparecer como valores atípicos, por lo que será importante considerarlos durante la preparación de los datos y el entrenamiento de los modelos. Esto es especialmente importante porque algunos algoritmos, como SVM, pueden verse afectados por variables con escalas muy amplias o valores extremos.

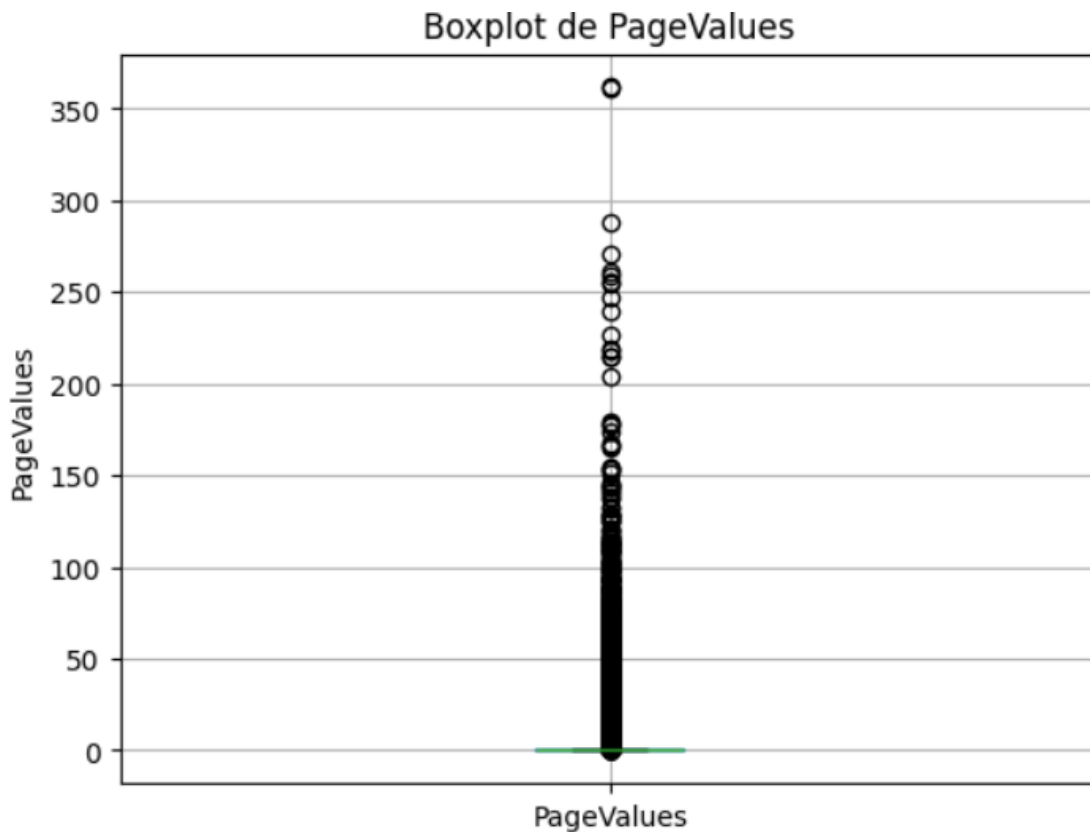
Figura 2.9 Histogramas de ExitRates



En la figura 2.9 se observa la distribución de la variable ExitRates, la cual representa la tasa de salida de las páginas visitadas durante una sesión. Esta variable permite analizar qué tan probable es que una página haya sido la última visitada antes de que el usuario abandonara el sitio.

La distribución permite identificar que existen sesiones con diferentes niveles de tasa de salida. Cuando los valores de ExitRates son altos, puede interpretarse que los usuarios abandonaron el sitio con mayor frecuencia después de visitar ciertas páginas. Este comportamiento puede ser importante para el problema de clasificación, ya que una tasa de salida elevada podría estar relacionada con una menor probabilidad de generar ingresos.

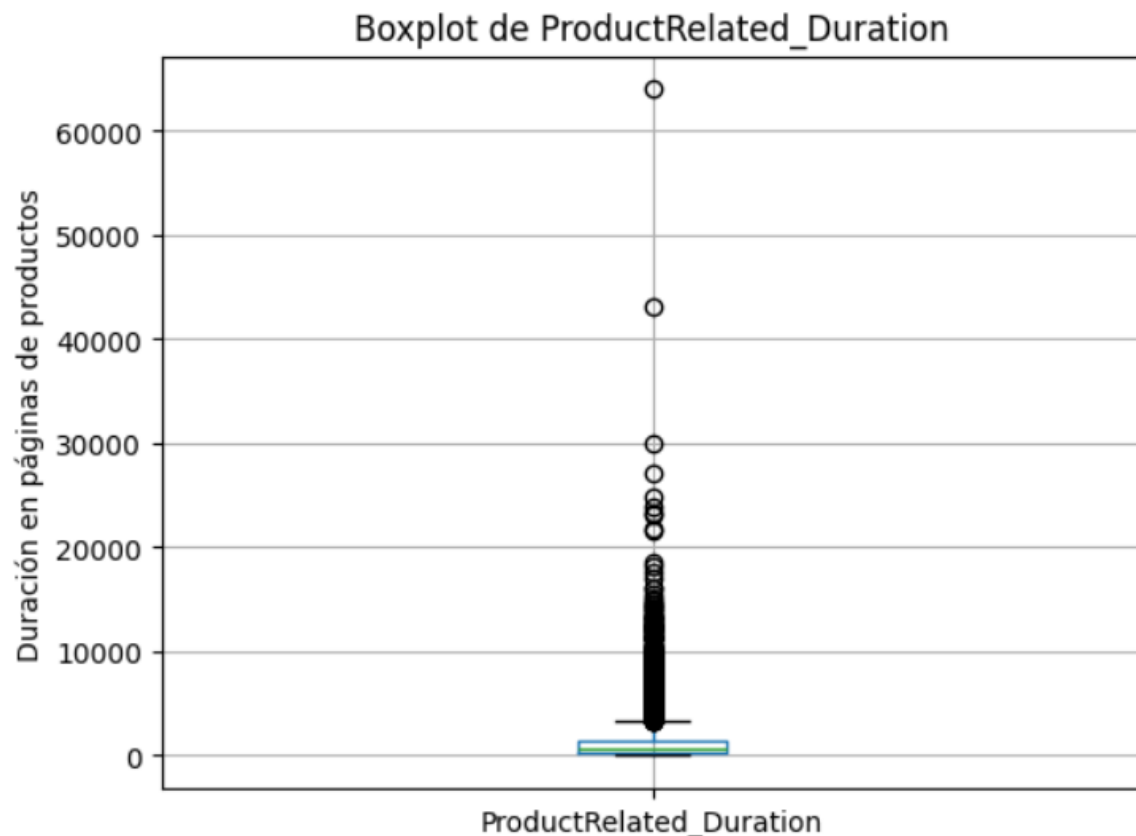
Figura 2.10 Boxplot de PageValues



En la figura 2.10 se observa la dispersión del valor estimado de las páginas visitadas durante las sesiones. La mayoría de los valores se concentran en una zona baja, mientras que también se pueden identificar valores más altos alejados del grupo principal de datos.

Estos valores altos pueden considerarse valores atípicos, pero no necesariamente deben interpretarse como errores. En este contexto, podrían representar sesiones donde el usuario visitó páginas con mayor valor antes de realizar una compra o acercarse a una transacción. Por ello, PageValues puede ser una variable relevante para el modelo, ya que puede aportar información relacionada con la intención de compra.

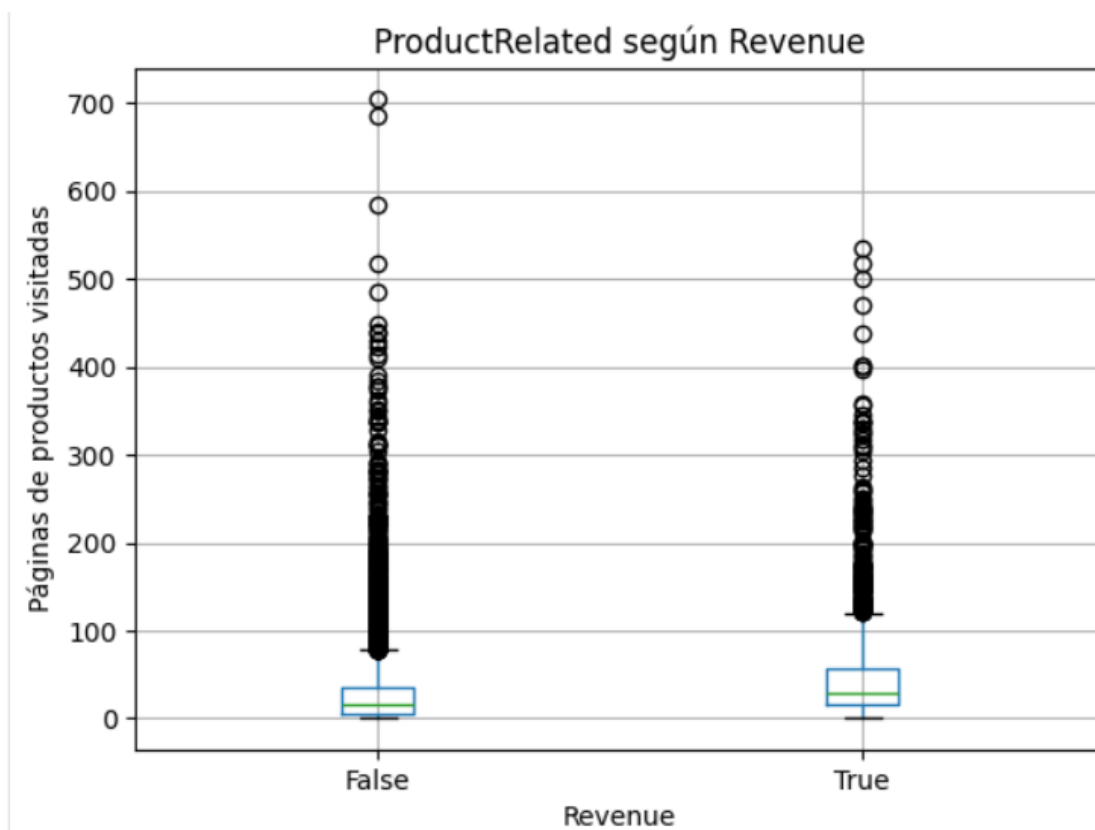
Figura 2.11 Boxplot de ProductRelated_Duration



En la figura 2.11 se observa la variabilidad del tiempo que los usuarios permanecieron en páginas relacionadas con productos. La gráfica muestra que la mayoría de las sesiones tienen una duración relativamente baja, pero también existen sesiones con tiempos mucho más altos.

La presencia de valores atípicos indica que algunos usuarios navegaron durante mucho más tiempo en páginas de productos. Estos casos pueden ser importantes porque reflejan sesiones con mayor interacción dentro de la tienda en línea. Sin embargo, también es necesario analizarlos con cuidado, ya que los valores extremos pueden influir en algunos modelos de clasificación, especialmente en aquellos sensibles a la escala de los datos, como SVM.

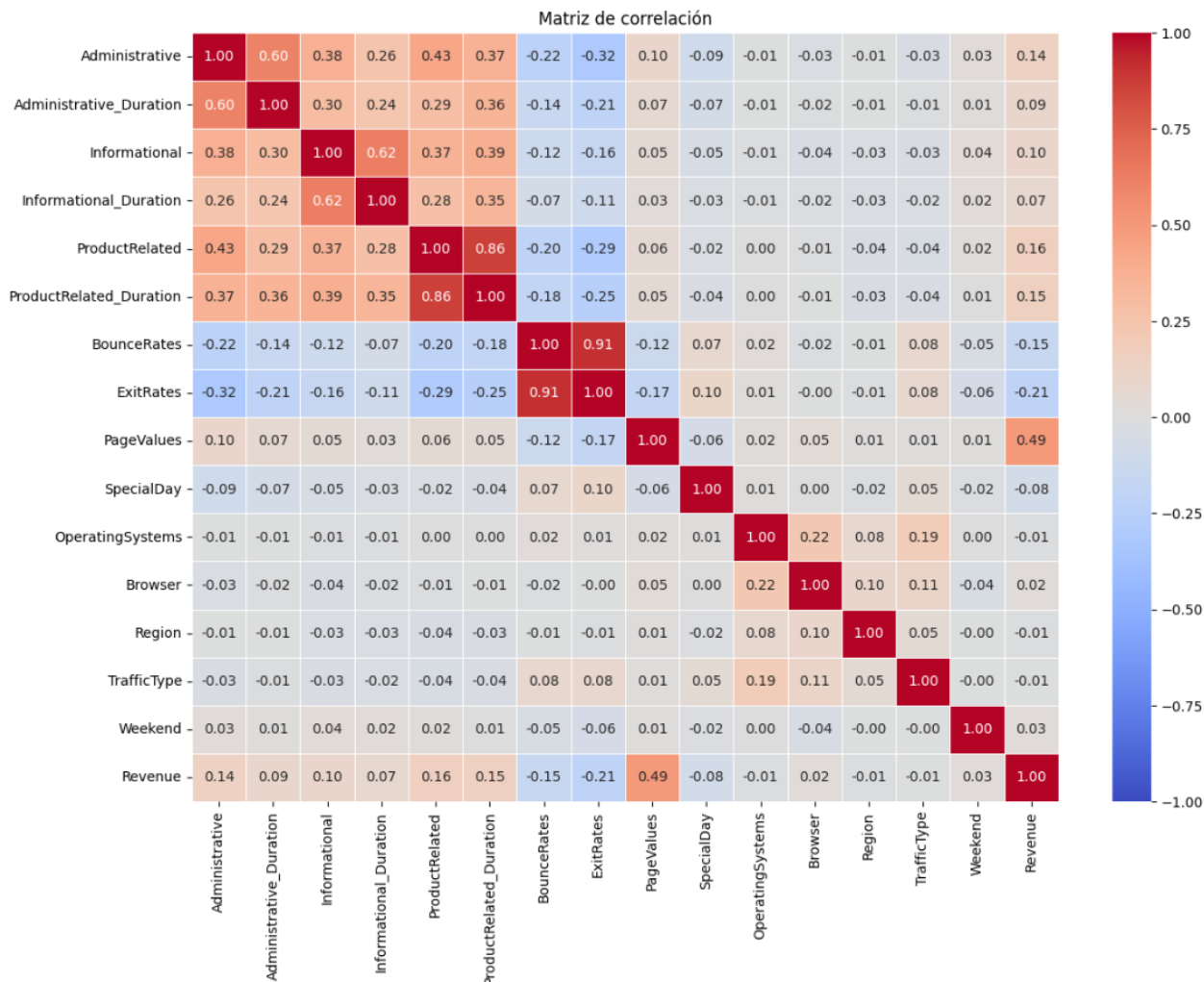
Figura 2.12 Boxplot de ProductRelated según Revenue



En la figura 2.12 se compara la cantidad de páginas de productos visitadas entre las sesiones que generaron ingresos y las que no generaron ingresos. Esta visualización permite observar si existen diferencias en el comportamiento de navegación de ambos grupos.

Si las sesiones con Revenue = True presentan valores más altos en ProductRelated, se puede interpretar que los usuarios que visitan más páginas de productos tienen mayor probabilidad de generar ingresos. Este patrón es relevante para el proyecto, ya que muestra una posible relación entre la interacción con productos y la intención de compra. Por lo tanto, esta variable puede ser útil para entrenar los modelos de clasificación.

Figura 2.13 Matriz de correlación



En la matriz de correlación se observan las relaciones entre las variables numéricas del dataset. Además de los colores, se muestran los valores numéricos de correlación, lo que permite identificar con mayor claridad qué variables tienen relaciones cercanas a 1, cercanas a -1 o cercanas a 0. Los valores positivos indican que ambas variables tienden a aumentar juntas, mientras que los valores negativos indican una relación inversa.

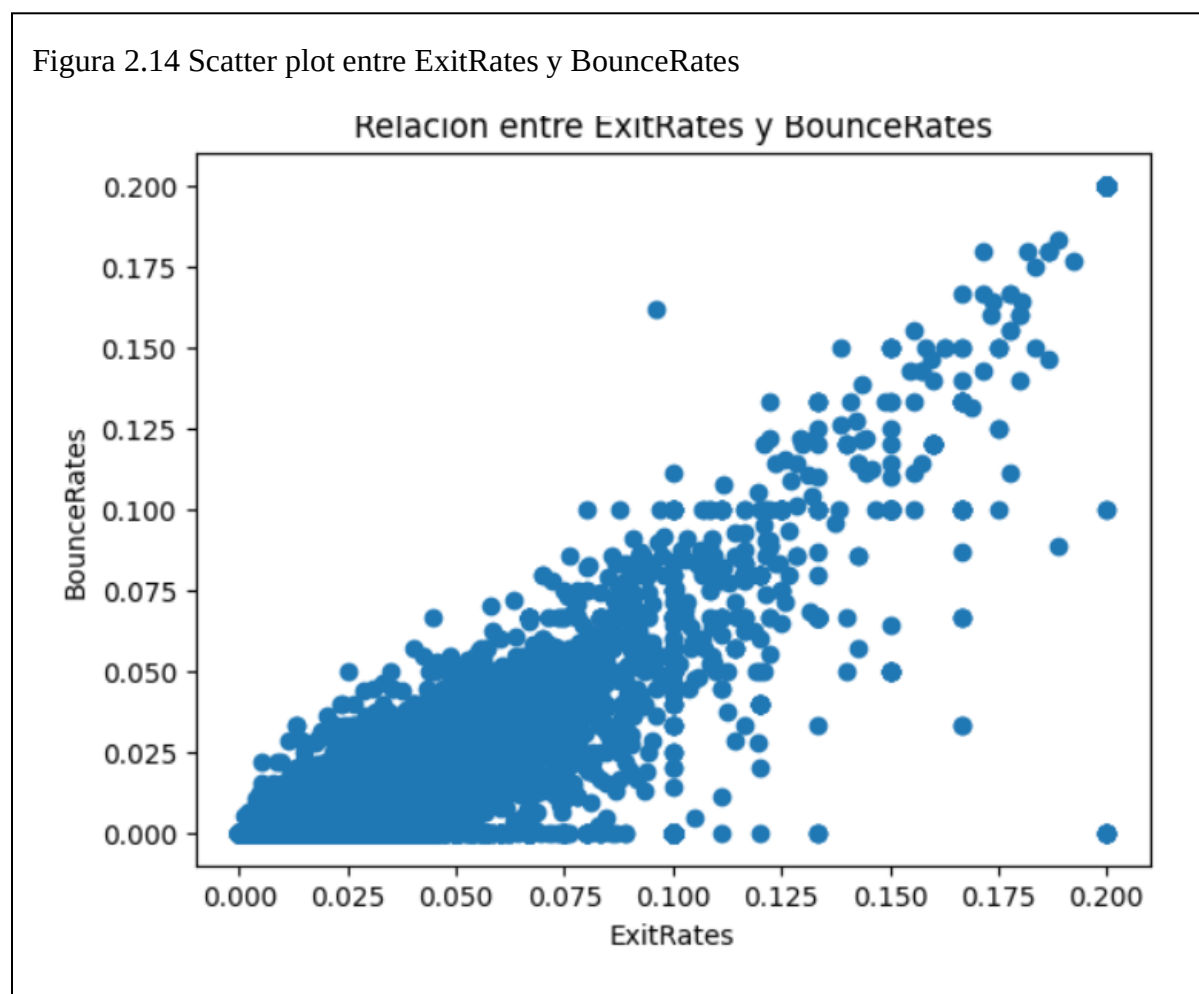
Esta visualización permite identificar posibles relaciones importantes entre variables del comportamiento de navegación. Por ejemplo, variables como BounceRates y ExitRates pueden

presentar una relación positiva debido a que ambas están asociadas con el abandono del sitio.

También pueden observarse relaciones entre variables como ProductRelated y

ProductRelated_Duration, ya que al visitar más páginas de productos es posible que aumente el tiempo total de navegación en esa categoría.

Este análisis ayuda a detectar variables que podrían ser útiles para el modelo de clasificación y permite observar qué atributos tienen mayor relación con la variable objetivo Revenue.



En el scatter plot se observa la relación entre ExitRates y BounceRates, dos variables relacionadas con el abandono del sitio web. BounceRates representa la tasa de rebote, mientras que ExitRates indica la frecuencia con la que una página fue la última visitada durante la sesión.

La gráfica permite observar si existe una tendencia entre ambas variables. Si los puntos muestran que valores altos de BounceRates también se relacionan con valores altos de ExitRates, se puede interpretar que ambas métricas están asociadas con sesiones donde el usuario abandona rápidamente el sitio. Este patrón es relevante porque puede estar relacionado con una menor intención de compra. Por lo tanto, estas variables pueden ser importantes para el modelo de clasificación, ya que ayudan a representar el comportamiento de abandono de los usuarios dentro de la tienda en línea. Una sesión con altas tasas de rebote y salida podría tener menor probabilidad de generar ingresos.

Figura 2.15 Código de distribución de visitas por mes

```
[35]: plt.figure(figsize=(15, 7))

# Gráfica 1: distribución de visitas por mes
plt.subplot(1, 2, 1)
plt.title("Distribución de sesiones por mes")

conteo_meses = df["Month"].value_counts()

plt.pie(
    x=conteo_meses,
    labels=conteo_meses.index,
    autopct="%.2f%%",
    shadow=True,
    explode=[0.1 if i < 3 else 0 for i in range(len(conteo_meses))]
)

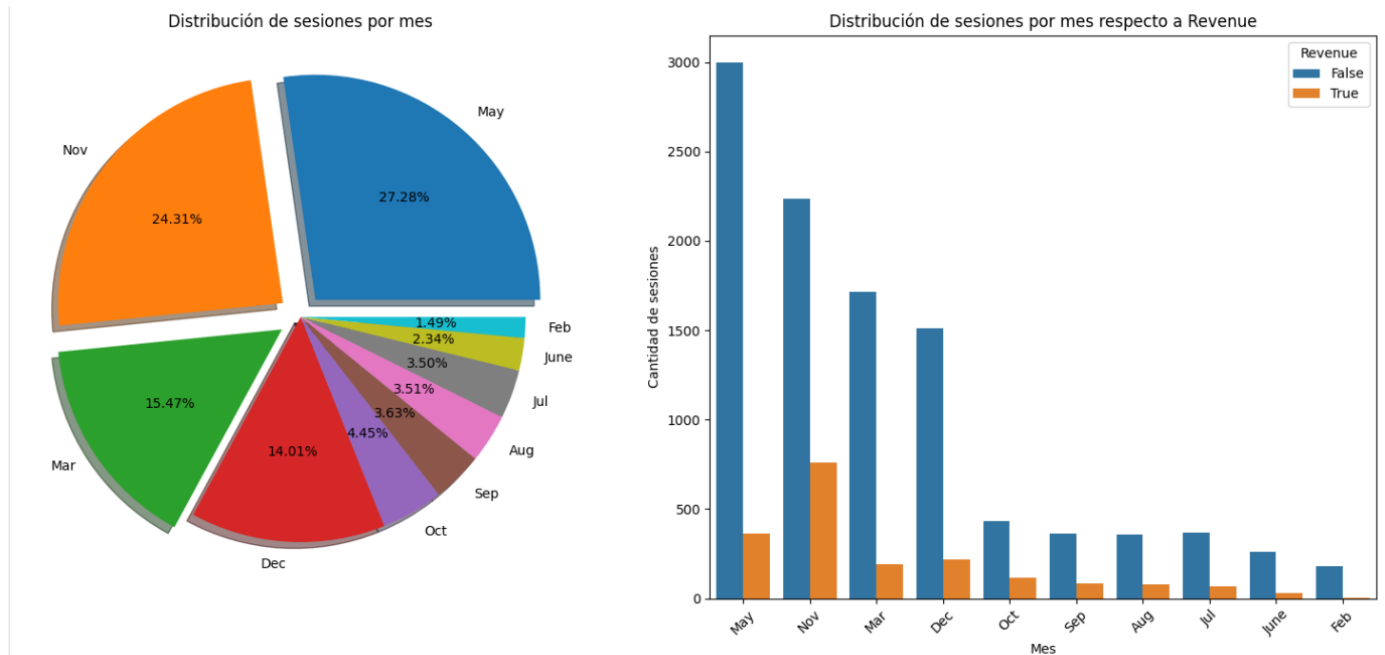
# Gráfica 2: distribución de meses respecto a Revenue
plt.subplot(1, 2, 2)
plt.title("Distribución de sesiones por mes respecto a Revenue")

sns.countplot(
    x="Month",
    hue="Revenue",
    data=df,
    order=conteo_meses.index
)

plt.xlabel("Mes")
plt.ylabel("Cantidad de sesiones")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

Figura 2.16 Grafica de distribución de visitas por mes



En la Figura 1.16 se muestra la distribución de las sesiones de usuarios según el mes en que ocurrieron. En la gráfica circular se observa que algunos meses concentran una mayor cantidad de visitas dentro del dataset, lo cual indica que la actividad de los usuarios no se distribuye de manera uniforme durante todo el año.

En la segunda gráfica se compara la variable Month con la variable objetivo Revenue. Esta visualización permite observar cuántas sesiones generaron ingresos y cuántas no generaron ingresos en cada mes. De manera general, se puede identificar que en la mayoría de los meses existen más sesiones con Revenue = False que con Revenue = True, lo cual coincide con el desbalance observado previamente en la variable objetivo.

Este análisis es útil porque permite identificar posibles patrones temporales en el comportamiento de compra. Algunos meses pueden presentar mayor cantidad de sesiones o una mayor proporción de sesiones que generan ingresos, lo cual podría relacionarse con temporadas comerciales, promociones, campañas de marketing o fechas especiales. Por esta razón, la variable Month puede ser relevante para el modelo de clasificación, ya que aporta información sobre el contexto temporal de la sesión.

Hallazgos relevantes del EDA

A partir del análisis exploratorio de datos se identificaron varios hallazgos importantes. En primer lugar, la variable objetivo Revenue presenta un posible desbalance entre clases, ya que la mayoría de las sesiones no generaron ingresos. Esto es relevante porque puede influir en el entrenamiento y evaluación de los modelos de clasificación.

También se observó que algunas variables presentan distribuciones sesgadas, como PageValues, donde muchos valores se concentran cerca de cero y pocos registros presentan valores altos. Este comportamiento indica que no todas las sesiones tienen el mismo valor para la tienda en línea y que algunas visitas pueden estar más relacionadas con una posible compra.

Los boxplots permiten identificar dispersión y posibles valores atípicos en variables como BounceRates y ProductRelated. Estos valores atípicos no necesariamente deben eliminarse de inmediato, ya que pueden representar comportamientos reales de los usuarios, como sesiones muy largas o visitas con alta interacción.

La matriz de correlación y los scatter plots permiten observar posibles relaciones entre variables numéricas. Aunque estas relaciones no prueban causalidad, sí ayudan a identificar atributos que podrían aportar información útil al modelo de clasificación. En general, variables

como PageValues, ProductRelated, ProductRelated_Duration, BounceRates y ExitRates parecen ser relevantes para analizar la intención de compra de los usuarios.

Estos hallazgos serán considerados en las siguientes etapas de la metodología CRISP-DM, especialmente durante la preparación de los datos y la selección de atributos para el entrenamiento de los modelos.

III. Preparación de los datos

Tratamiento de valores faltantes.

En esta etapa se realizó la revisión y tratamiento de valores faltantes dentro del dataset. Primero, se analizó el conjunto de datos original mediante la función `df.isnull().sum()`, con la finalidad de identificar si existían valores nulos en alguna de sus columnas.

El resultado obtenido fue igual a cero, lo cual confirma que el dataset original no contiene valores faltantes. Sin embargo, debido a que la actividad solicita realizar un tratamiento de valores faltantes, se generaron valores nulos de manera aleatoria en una copia del dataset original. Esta decisión permitió cumplir con el requisito de la actividad sin modificar directamente la versión inicial de los datos.

Para realizar este procedimiento se creó una copia llamada `df_nulos`. Posteriormente, se seleccionaron algunas variables predictoras en las que se insertaron valores faltantes de forma controlada. Las columnas seleccionadas fueron PageValues, ProductRelated_Duration, BounceRates, ExitRates, Month y VisitorType.

Figura 3.1 Generación aleatoria de valores nulos

```
[32]: # Crear una copia del dataset original
df_nulos = df.copy()
# Fijar semilla para que el resultado sea reproducible
np.random.seed(42)
# Columnas donde se generarán valores faltantes
columnas_con_nulos = [
    "PageValues",
    "ProductRelated_Duration",
    "BounceRates",
    "ExitRates",
    "Month",
    "VisitorType"
]
# Porcentaje de valores nulos a generar
porcentaje_nulos = 0.02
# Generar valores nulos aleatoriamente
for columna in columnas_con_nulos:
    indices = df_nulos.sample(
        frac=porcentaje_nulos,
        random_states=42
    ).index
    df_nulos.loc[indices, columna] = np.nan
# Mostrar valores nulos generados en las columnas seleccionadas
df_nulos[columnas_con_nulos].isnull().sum()

[32]: PageValues          247
ProductRelated_Duration  247
BounceRates              247
ExitRates                247
Month                   247
VisitorType              247
dtype: int64
```

Y volvemos a mostrar los nulos pero en este caso con el `df_nulos.isnull().sum()`.

Figura 3.2 Revisión de nulos de la copia del dataset.

```
df_nulos.isnull().sum()
```

```
Administrative          0
Administrative_Duration  0
Informational           0
Informational_Duration  0
ProductRelated          0
ProductRelated_Duration 247
BounceRates             247
ExitRates               247
PageValues              247
SpecialDay              0
Month                  247
OperatingSystems        0
Browser                 0
Region                  0
TrafficType             0
VisitorType             247
Weekend                 0
Revenue                 0
dtype: int64
```

Las columnas fueron seleccionadas porque incluyen tanto variables numéricas como categóricas. Esto permite demostrar dos formas distintas de tratamiento de valores faltantes. Además, se evitó generar valores nulos en la variable objetivo Revenue, ya que esta representa la clase que los modelos deberán aprender a predecir.

Se generó aproximadamente un 2% de valores nulos en cada una de las columnas seleccionadas. Este porcentaje se consideró adecuado porque permite simular la presencia de datos faltantes sin alterar de manera excesiva la estructura general del dataset. También se utilizó una semilla aleatoria mediante `np.random.seed(42)`, con el propósito de que el procedimiento fuera reproducible.

Después de generar los valores faltantes, se aplicó una técnica de imputación. Para las variables numéricas PageValues, ProductRelated_Duration, BounceRates y ExitRates, se utilizó la mediana. Esta decisión se justifica porque durante el análisis exploratorio se observó que algunas variables presentan distribuciones sesgadas y valores atípicos. En estos casos, la mediana es más adecuada que la media, ya que no se ve tan afectada por valores extremos.

Para las variables categóricas Month y VisitorType, se utilizó la moda. Esta técnica consiste en reemplazar los valores faltantes con la categoría más frecuente de cada columna. La moda es adecuada para variables categóricas porque no es posible calcular media o mediana sobre valores textuales.

Figura 3.3 Tratamiento de valores nulos por imputación

```
: # Crear una copia para aplicar el tratamiento de valores faltantes
df_tratado = df_nulos.copy()

# Columnas numéricas con valores nulos
columnas_numericas_nulos = [
    "PageValues",
    "ProductRelated_Duration",
    "BounceRates",
    "ExitRates"
]

# Columnas categóricas con valores nulos
columnas_categoricas_nulos = [
    "Month",
    "VisitorType"
]

# Imputar variables numéricas con la mediana
for columna in columnas_numericas_nulos:
    mediana = df_tratado[columna].median()
    df_tratado[columna] = df_tratado[columna].fillna(mediana)

# Imputar variables categóricas con la moda
for columna in columnas_categoricas_nulos:
    moda = df_tratado[columna].mode()[0]
    df_tratado[columna] = df_tratado[columna].fillna(moda)

# Verificar que ya no existan valores nulos
df_tratado.isnull().sum()

: Administrative          0
  Administrative_Duration  0
  Informational           0
  Informational_Duration  0
  ProductRelated          0
  ProductRelated_Duration 0
  BounceRates             0
  ExitRates               0
  PageValues              0
  SpecialDay              0
  Month                   0
  OperatingSystems        0
  Browser                 0
  Region                  0
  TrafficType             0
  VisitorType             0
  Weekend                 0
  Revenue                 0
dtype: int64
```

No se eligió eliminar filas con valores faltantes porque los nulos fueron generados artificialmente para cumplir con el ejercicio. Además, eliminar registros reduciría innecesariamente la cantidad de datos disponibles para el entrenamiento de los modelos. Al

tratarse de un porcentaje bajo de valores faltantes, la imputación permite conservar la información del dataset y mantener su tamaño original.

Finalmente, se verificó nuevamente la existencia de valores nulos mediante `df_tratado.isnull().sum().sum()`.

Figura 3.4 Confirmación final del total de nulos

```
dtype: int64  
[34]: df_tratado.isnull().sum().sum()  
[34]: np.int64(0)
```

El resultado final fue igual a cero, lo que confirma que el tratamiento aplicado eliminó correctamente los valores faltantes. Con esto, el dataset quedó preparado para continuar con la conversión de variables categóricas, el escalado de variables numéricas y la selección de atributos.

Conversión de variables categóricas a numéricas

En esta etapa se identificaron las variables categóricas del dataset con el propósito de transformarlas a formato numérico. Esta conversión es necesaria porque los modelos de clasificación, como SVM y Random Forest, requieren que las variables de entrada estén representadas mediante valores numéricos para poder entrenarse correctamente.

Las variables categóricas identificadas fueron Month, VisitorType, Weekend, OperatingSystems, Browser, Region y TrafficType. Aunque algunas de estas variables aparecen en Python como valores enteros, como OperatingSystems, Browser, Region y TrafficType, se consideran categóricas porque representan códigos o clases, no cantidades numéricas continuas.

Por ejemplo, un navegador con código 3 no significa que sea mayor que un navegador con código 2; simplemente representa una categoría distinta.

Figura 3.5 Identificación de variables categóricas

```
# Identificar variables categóricas que requieren transformación
columnas_categoricas = [
    "Month",
    "VisitorType",
    "Weekend",
    "OperatingSystems",
    "Browser",
    "Region",
    "TrafficType"
]

updated_df[columnas_categoricas].head()
```

	Month	VisitorType	Weekend	OperatingSystems	Browser	Region	TrafficType
0	Feb	Returning_Visitor	False	1	1	1	1
1	Feb	Returning_Visitor	False	2	2	1	2
2	Feb	Returning_Visitor	False	4	1	9	3
3	Feb	Returning_Visitor	False	3	2	2	4
4	Feb	Returning_Visitor	True	3	3	1	4

Posteriormente, se revisó la cantidad de valores únicos en cada variable categórica. Esta revisión permitió conocer cuántas categorías distintas tenía cada atributo antes de aplicar la transformación.

Figura 3.6 Revisar valores únicos de las variables categóricas

```
# Revisar la cantidad de categorías únicas en cada variable categórica
updated_df[columnas_categoricas].nunique()
```

Month	10
VisitorType	3
Weekend	2
OperatingSystems	8
Browser	13
Region	9
TrafficType	20
dtype:	int64

Para realizar la conversión se utilizó la técnica de **One-Hot Encoding**, implementada mediante la función `pd.get_dummies()`. Esta técnica crea nuevas columnas binarias para representar las categorías de cada variable. Por ejemplo, en lugar de conservar una columna textual como `Month`, se generan columnas que indican la presencia o ausencia de cada mes mediante valores 0 y 1.

Se eligió One-Hot Encoding porque las variables categóricas del dataset son principalmente nominales, es decir, no tienen un orden natural. Por esta razón, no se utilizó `LabelEncoder`, ya que este método asigna números consecutivos a las categorías y podría generar una falsa relación de orden entre ellas. Esto podría afectar la interpretación de los modelos, especialmente en algoritmos sensibles a la escala o distancia entre valores.

Figura 3.7 Conversión con One-Hot Encoding

```
[79]: # Aplicar One-Hot Encoding a las variables categóricas
updated_df_encoded = pd.get_dummies(
    updated_df,
    columns=columnas_categoricas,
    drop_first=True,
    dtype=int
)
updated_df_encoded.head()
```

	Administrative	Administrative_Duration	Informational	Informational_Duration	ProductRelated	ProductRelated_Duration	BounceRates	ExitRates	PageValues	Spec
0	0	0.0	0	0.0	1	0.000000	0.20	0.20	0.0	
1	0	0.0	0	0.0	2	64.000000	0.00	0.10	0.0	
2	0	0.0	0	0.0	1	0.000000	0.20	0.20	0.0	
3	0	0.0	0	0.0	2	2.666667	0.05	0.14	0.0	
4	0	0.0	0	0.0	10	627.500000	0.02	0.05	0.0	

5 rows × 69 columns

Además, la variable objetivo Revenue, que originalmente estaba representada con valores booleanos (True y False), fue convertida a formato numérico, donde 1 representa que la sesión generó ingresos y 0 representa que la sesión no generó ingresos. Esta conversión permite que los modelos de clasificación puedan trabajar correctamente con la variable dependiente.

Después de aplicar la transformación, se verificaron los tipos de datos mediante `updated_df_encoded.info()`. Esta revisión permitió confirmar que las variables categóricas fueron convertidas correctamente a columnas numéricas.

Figura 3.7 Convertir la variable objetivo Revenue a formato numérico

```
# Convertir la variable objetivo Revenue de True/False a 1/0
updated_df_encoded["Revenue"] = updated_df_encoded["Revenue"].astype(int)

updated_df_encoded["Revenue"].value_counts()
```

```
Revenue
0    10422
1     1908
Name: count, dtype: int64
```


Finalmente, se compararon las dimensiones del dataset antes y después de aplicar One-Hot Encoding. Como resultado, el número de columnas aumentó, lo cual es normal en esta técnica, ya que cada categoría se representa mediante una nueva columna binaria.

Figura 3.7 Comparar dimensiones antes y después

```
print("Dimensiones antes de la conversión:", updated_df.shape)
print("Dimensiones después de la conversión:", updated_df_encoded.shape)

Dimensiones antes de la conversión: (12330, 18)
Dimensiones después de la conversión: (12330, 69)
```

Con esta transformación, el dataset quedó preparado para continuar con la etapa de escalado o normalización de variables numéricas.

Escalado o normalización.

En esta etapa se aplicó escalado a las variables numéricas del dataset con el propósito de que los datos quedaran en una escala comparable antes de entrenar los modelos de clasificación. Este paso es importante porque algunas variables presentan rangos muy diferentes entre sí. Por ejemplo, variables como BounceRates, ExitRates y SpecialDay manejan valores pequeños, mientras que variables como ProductRelated_Duration, Administrative_Duration o PageValues pueden presentar valores mucho más altos.

Para realizar el escalado, primero se separó la variable objetivo Revenue de las variables independientes. Esto se hizo porque Revenue representa la clase que se desea predecir y no debe ser transformada como una variable predictora. Posteriormente, los datos se dividieron en conjunto de entrenamiento y conjunto de prueba, utilizando un 80% para entrenamiento y un 20% para prueba.

Figura 3.8 Separación de X, y y división train/test

```
# Separar variables independientes y variable dependiente  
X = updated_df_encoded.drop("Revenue", axis=1)  
y = updated_df_encoded["Revenue"]
```

Después de dividir los datos, se seleccionaron únicamente las variables numéricas continuas para aplicar el escalado. Las variables seleccionadas fueron Administrative, Administrative_Duration, Informational, Informational_Duration, ProductRelated, ProductRelated_Duration, BounceRates, ExitRates, PageValues y SpecialDay.

Figura 3.9. Selección de variables numéricas para escalado

```
# Columnas numéricas continuas que serán escaladas  
columnas_a_escalar = [  
    "Administrative",  
    "Administrative_Duration",  
    "Informational",  
    "Informational_Duration",  
    "ProductRelated",  
    "ProductRelated_Duration",  
    "BounceRates",  
    "ExitRates",  
    "PageValues",  
    "SpecialDay"  
]
```

La técnica utilizada fue **StandardScaler**, la cual transforma los datos para que tengan una media cercana a 0 y una desviación estándar cercana a 1. Esta técnica fue seleccionada porque uno de los modelos que se utilizará en el proyecto es SVM, el cual es sensible a la escala

de las variables. Si las variables no se escalan, aquellas con valores más grandes podrían influir más en el modelo, aunque no necesariamente sean más importantes para la clasificación.

Figura 3.9. Aplicación de StandardScaler a las variables numéricas.

```
# Crear copias para no modificar directamente X_train y X_test
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

# Aplicar StandardScaler
scaler = StandardScaler()

X_train_scaled[columnas_a_escalarse] = scaler.fit_transform(X_train[columnas_a_escalarse])
X_test_scaled[columnas_a_escalarse] = scaler.transform(X_test[columnas_a_escalarse])

X_train_scaled.head()
```

Es importante mencionar que el escalador se ajustó únicamente con los datos de entrenamiento mediante `fit_transform()`, mientras que en los datos de prueba se aplicó solamente `transform()`. Esto evita que la información del conjunto de prueba influya en el proceso de entrenamiento, lo cual ayuda a prevenir fuga de información.

Las variables categóricas convertidas mediante One-Hot Encoding no fueron escaladas, ya que quedaron representadas con valores binarios 0 y 1. Estas columnas indican la ausencia o presencia de una categoría, por lo que se conservaron en su forma codificada.

Figura 3.10. Datos de entrenamiento después del escalado.

	Administrative	Administrative_Duration	Informational	Informational_Duration	ProductRelated	ProductRelated_Duration	BounceRates	ExitRates	PageValues
4263	1.712088	3.624745	-0.395782	-0.244589	0.058405	0.092416	-0.349454	-0.611748	0.349986
5905	-0.698294	-0.452341	-0.395782	-0.244589	-0.629459	-0.536130	0.369901	1.173202	-0.316754
9434	-0.698294	-0.452341	-0.395782	-0.244589	-0.629459	-0.597051	-0.455241	0.143321	-0.316754
3505	-0.095698	1.429605	-0.395782	-0.244589	-0.331385	0.018279	-0.197384	-0.114149	0.728168
2067	-0.698294	-0.452341	-0.395782	-0.244589	0.012548	-0.181577	-0.455241	-0.874480	2.653291

Finalmente, se revisaron las estadísticas descriptivas de las variables escaladas para comprobar que la transformación se realizó correctamente. Después del escalado, las variables numéricas quedaron en una escala más uniforme, lo cual permite que los modelos de clasificación trabajen de manera más adecuada.

Figura 3.11. Estadísticas descriptivas después del escalado.

```
# Estadísticas después del escalado
```

```
X_train_scaled[columnas_a_escalas].describe().T
```

	count	mean	std	min	25%	50%	75%	max
Administrative	9864.0	-4.610172e-17	1.000051	-0.698294	-0.698294	-0.396996	0.506897	7.436746
Administrative_Duration	9864.0	-1.872882e-17	1.000051	-0.452341	-0.452341	-0.407798	0.065472	18.471518
Informational	9864.0	-2.161018e-18	1.000051	-0.395782	-0.395782	-0.395782	-0.395782	18.426209
Informational_Duration	9864.0	2.161018e-17	1.000051	-0.244589	-0.244589	-0.244589	-0.244589	17.782157
ProductRelated	9864.0	4.358053e-17	1.000051	-0.721175	-0.560673	-0.308456	0.150121	15.443645
ProductRelated_Duration	9864.0	2.881357e-17	1.000051	-0.619262	-0.519643	-0.302607	0.139303	33.211941
BounceRates	9864.0	4.322036e-18	1.000051	-0.455241	-0.455241	-0.391769	-0.111432	3.670470
ExitRates	9864.0	-6.122884e-17	1.000051	-0.886561	-0.592309	-0.365793	0.143321	3.232965
PageValues	9864.0	-1.656781e-17	1.000051	-0.316754	-0.316754	-0.316754	-0.316754	19.334425
SpecialDay	9864.0	-7.473521e-17	1.000051	-0.311499	-0.311499	-0.311499	-0.311499	4.657175

Selección de atributos

La variable X se construyó eliminando la columna Revenue del dataset preparado, ya que esta columna representa el resultado que se desea predecir y no debe formar parte de las variables de entrada. Por otro lado, la variable y contiene únicamente los valores de Revenue, los cuales serán utilizados como etiquetas de clase durante el entrenamiento y evaluación de los modelos.

Figura 3.12. Visualización de variables independientes

```
# Mostrar las primeras filas de las variables independientes
X.head()
```

	Administrative	Administrative_Duration	Informational	Informational_Duration	ProductRelated	ProductRelated_Duration	BounceRates	ExitRates	PageValues	Speci
0	0	0.0	0	0.0	1	0.000000	0.20	0.20	0.0	
1	0	0.0	0	0.0	2	64.000000	0.00	0.10	0.0	
2	0	0.0	0	0.0	1	0.000000	0.20	0.20	0.0	
3	0	0.0	0	0.0	2	2.666667	0.05	0.14	0.0	
4	0	0.0	0	0.0	10	627.500000	0.02	0.05	0.0	

5 rows × 68 columns

La selección de los atributos incluidos en X se justifica porque todos aportan información potencialmente útil para identificar patrones relacionados con la intención de compra. Las variables como ProductRelated, ProductRelated_Duration, PageValues, BounceRates y ExitRates describen el comportamiento del usuario dentro del sitio web. Por su parte, las variables transformadas mediante One-Hot Encoding, como Month, VisitorType, OperatingSystems, Browser, Region, TrafficType y Weekend, aportan información categórica sobre el contexto de la sesión.

Figura 3.13. Distribución de la variable dependiente Revenue

```
# Mostrar distribución de la variable dependiente
y.value_counts()

Revenue
0    10422
1     1908
Name: count, dtype: int64
```

En este proyecto no se descartaron atributos predictivos completos del dataset original, ya que no existen columnas identificadoras como folios, nombres de usuarios o identificadores únicos que pudieran generar ruido o no aportar información al modelo. Sin embargo, las variables categóricas originales fueron reemplazadas por columnas binarias mediante One-Hot Encoding. Además, al utilizar `drop_first=True`, se eliminó una categoría de referencia por cada variable categórica, con el propósito de evitar redundancia entre columnas.

Figura 3.14. Atributos seleccionados como variables independientes

```
# Mostrar nombres de columnas utilizadas como variables independientes
X.columns

Index(['Administrative', 'Administrative_Duration', 'Informational',
      'Informational_Duration', 'ProductRelated', 'ProductRelated_Duration',
      'BounceRates', 'ExitRates', 'PageValues', 'SpecialDay', 'Month_Dec',
      'Month_Feb', 'Month_Jul', 'Month_June', 'Month_Mar', 'Month_May',
      'Month_Nov', 'Month_Oct', 'Month_Sep', 'VisitorType_Other',
      'VisitorType_Returning_Visitor', 'Weekend_True', 'OperatingSystems_2',
      'OperatingSystems_3', 'OperatingSystems_4', 'OperatingSystems_5',
      'OperatingSystems_6', 'OperatingSystems_7', 'OperatingSystems_8',
      'Browser_2', 'Browser_3', 'Browser_4', 'Browser_5', 'Browser_6',
      'Browser_7', 'Browser_8', 'Browser_9', 'Browser_10', 'Browser_11',
      'Browser_12', 'Browser_13', 'Region_2', 'Region_3', 'Region_4',
      'Region_5', 'Region_6', 'Region_7', 'Region_8', 'Region_9',
      'TrafficType_2', 'TrafficType_3', 'TrafficType_4', 'TrafficType_5',
      'TrafficType_6', 'TrafficType_7', 'TrafficType_8', 'TrafficType_9',
      'TrafficType_10', 'TrafficType_11', 'TrafficType_12', 'TrafficType_13',
      'TrafficType_14', 'TrafficType_15', 'TrafficType_16', 'TrafficType_17',
      'TrafficType_18', 'TrafficType_19', 'TrafficType_20'],
      dtype='str')
```

IV. Modelado

En esta etapa se entrenaron dos modelos de clasificación utilizando el dataset previamente preparado. Antes del modelado, los datos fueron tratados para eliminar valores faltantes, convertir variables categóricas a numéricas y escalar las variables numéricas. De esta manera, el conjunto de datos quedó listo para ser utilizado por los algoritmos de clasificación.

Los modelos seleccionados fueron una Máquina de Vectores de Soporte, SVM, y Random Forest. El modelo SVM fue incluido porque es obligatorio dentro de la actividad, mientras que Random Forest fue seleccionado como segundo algoritmo de clasificación debido a que permite trabajar con relaciones no lineales, es robusto ante valores atípicos y suele obtener buenos resultados en problemas de clasificación.

Modelo SVM

La Máquina de Vectores de Soporte, conocida como SVM, es un algoritmo de clasificación que busca encontrar una frontera de separación entre las clases. Su objetivo es construir un hiperplano que separe los datos de la mejor manera posible, maximizando el margen entre las clases. En este proyecto, SVM se utiliza para clasificar si una sesión de usuario generó ingresos o no, tomando como base las variables predictoras del dataset.

Para este modelo se utilizó el kernel `rbf`, ya que permite trabajar con relaciones no lineales entre las variables. También se utilizó `C=1.0`, que es un valor estándar para controlar el equilibrio entre margen y errores de clasificación. El parámetro `gamma="scale"` permite que el modelo ajuste automáticamente el alcance de influencia de los puntos de entrenamiento. Además, se utilizó `class_weight="balanced"` debido a que en el análisis exploratorio se observó un desbalance entre las clases de la variable `Revenue`.

Figura 4.1. Entrenamiento del modelo SVM

```
# Crear el modelo SVM
modelo_svm = SVC(
    kernel="rbf",
    C=1.0,
    gamma="scale",
    class_weight="balanced",
    probability=True,
    random_state=42
)

# Medir tiempo de entrenamiento
inicio_svm = time.time()

# Entrenar el modelo
modelo_svm.fit(X_train_scaled, y_train)

fin_svm = time.time()

# Calcular tiempo total
tiempo_svm = fin_svm - inicio_svm

print("Modelo SVM entrenado correctamente")
print("Tiempo de entrenamiento SVM:", tiempo_svm, "segundos")
```

El modelo SVM fue seleccionado porque es adecuado para problemas de clasificación binaria y porque permite construir fronteras de decisión complejas. Sin embargo, este algoritmo puede ser sensible a la escala de los datos, por lo que previamente se aplicó `StandardScaler` a las variables numéricas.

Figura 4.2. Tiempo de entrenamiento del modelo SVM

```
Modelo SVM entrenado correctamente
Tiempo de entrenamiento SVM: 48.62113928794861 segundos
```


Modelo Random Forest

El segundo modelo utilizado fue Random Forest. Este algoritmo se basa en la construcción de varios árboles de decisión, donde cada árbol realiza una predicción y el resultado final se obtiene mediante votación. En problemas de clasificación, Random Forest combina las predicciones de múltiples árboles para obtener una clasificación más estable y reducir el riesgo de sobreajuste que podría presentarse en un solo árbol de decisión.

Para este modelo se utilizaron 100 árboles mediante el parámetro `n_estimators=100`. También se utilizó el criterio `gini`, que permite medir la calidad de las divisiones dentro de cada árbol. El parámetro `random_state=42` se estableció para que los resultados sean reproducibles. Además, se utilizó `class_weight="balanced"` para considerar el desbalance existente entre las clases de la variable objetivo `Revenue`.

Figura 4.3. Entrenamiento del modelo Random Forest

```
# Crear el modelo Random Forest
modelo_rf = RandomForestClassifier(
    n_estimators=100,
    criterion="gini",
    max_depth=None,
    random_state=42,
    class_weight="balanced",
    n_jobs=-1
)

# Medir tiempo de entrenamiento
inicio_rf = time.time()

# Entrenar el modelo
modelo_rf.fit(X_train_scaled, y_train)

fin_rf = time.time()

# Calcular tiempo total
tiempo_rf = fin_rf - inicio_rf

print("Modelo Random Forest entrenado correctamente")
print("Tiempo de entrenamiento Random Forest:", tiempo_rf, "segundos")
```

Random Forest fue seleccionado como segundo algoritmo porque es un modelo robusto, puede trabajar con relaciones no lineales y permite capturar patrones complejos dentro de los datos. Además, es útil para comparar contra SVM, ya que ambos modelos tienen formas diferentes de aprender: SVM busca una frontera de separación entre clases, mientras que Random Forest combina múltiples árboles de decisión.

Figura 4.3. Entrenamiento del modelo Random Forest

```
Modelo Random Forest entrenado correctamente  
Tiempo de entrenamiento Random Forest: 0.718268871307373 segundos
```

Comparación inicial de los modelos entrenados

Después de entrenar ambos modelos, se registró el tiempo de entrenamiento de cada uno. Esta comparación es importante porque permite observar cuál modelo requiere más tiempo para ajustarse a los datos. Aunque el desempeño final se analizará en la etapa de evaluación, el tiempo de entrenamiento ya permite identificar diferencias de complejidad entre SVM y Random Forest.

Figura 4.5. Comparación de tiempos de entrenamiento entre modelos

```
Tiempo de entrenamiento SVM: 48.62113928794861 segundos  
Tiempo de entrenamiento Random Forest: 0.718268871307373 segundos
```

Nota. Random Forest presentó un menor tiempo de entrenamiento en comparación con SVM, aunque el rendimiento final será determinado en la etapa de evaluación.

Después de entrenar ambos modelos, se comparó el tiempo de entrenamiento de SVM y Random Forest. El modelo SVM obtuvo un tiempo de entrenamiento aproximado de **48.62 segundos**, mientras que Random Forest obtuvo un tiempo aproximado de **0.71 segundos**.

Estos resultados muestran que, en este caso, Random Forest fue más rápido de entrenar que SVM. Esto puede deberse a que SVM, especialmente con kernel rbf, requiere calcular

relaciones entre los datos para encontrar una frontera de separación adecuada entre las clases. Además, al utilizar `probability=True`, el entrenamiento de SVM puede volverse más costoso computacionalmente.

Por otro lado, Random Forest entrenó en menor tiempo porque construye varios árboles de decisión y puede aprovechar mejor el procesamiento paralelo mediante el parámetro `n_jobs=-1`. Aunque el tiempo de entrenamiento no determina por sí solo cuál modelo es mejor, sí permite comparar la complejidad práctica de ambos algoritmos. La calidad final de cada modelo será analizada posteriormente mediante métricas de evaluación como accuracy, precision, recall, F1-score, matriz de confusión y curva ROC.

V. Evaluación

En esta etapa se evaluó el desempeño de los dos modelos de clasificación entrenados: SVM y Random Forest. El objetivo de esta fase es comparar los resultados obtenidos por cada algoritmo y determinar cuál tuvo mejor rendimiento para predecir la variable Revenue.

Para evaluar los modelos se utilizaron diferentes métricas, entre ellas accuracy con k-fold cross-validation, matriz de confusión, precision, recall, F1-score, curvas ROC, AUC y tiempo de entrenamiento. Estas métricas permiten analizar el desempeño de los modelos desde diferentes perspectivas, ya que no basta con observar únicamente el accuracy cuando existe desbalance entre clases.

Accuracy usando k-fold cross-validation

Primero se aplicó k-fold cross-validation con 5 particiones utilizando StratifiedKFold. Esta variante de k-fold conserva la proporción de las clases en cada partición, lo cual es importante porque durante el análisis exploratorio se observó que la variable Revenue presenta desbalance entre las clases.

Figura 5.1. Accuracy con k-fold cross-validation para SVM y Random Forest

```
# Configurar k-fold estratificado
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Accuracy con k-fold para SVM
accuracy_svm_kfold = cross_val_score(
    modelo_svm,
    X_train_scaled,
    y_train,
    cv=kfold,
    scoring="accuracy"
)

# Accuracy con k-fold para Random Forest
accuracy_rf_kfold = cross_val_score(
    modelo_rf,
    X_train_scaled,
    y_train,
    cv=kfold,
    scoring="accuracy"
)

print("Accuracy SVM por fold:", accuracy_svm_kfold)
print("Accuracy promedio SVM:", accuracy_svm_kfold.mean())

print("Accuracy Random Forest por fold:", accuracy_rf_kfold)
print("Accuracy promedio Random Forest:", accuracy_rf_kfold.mean())

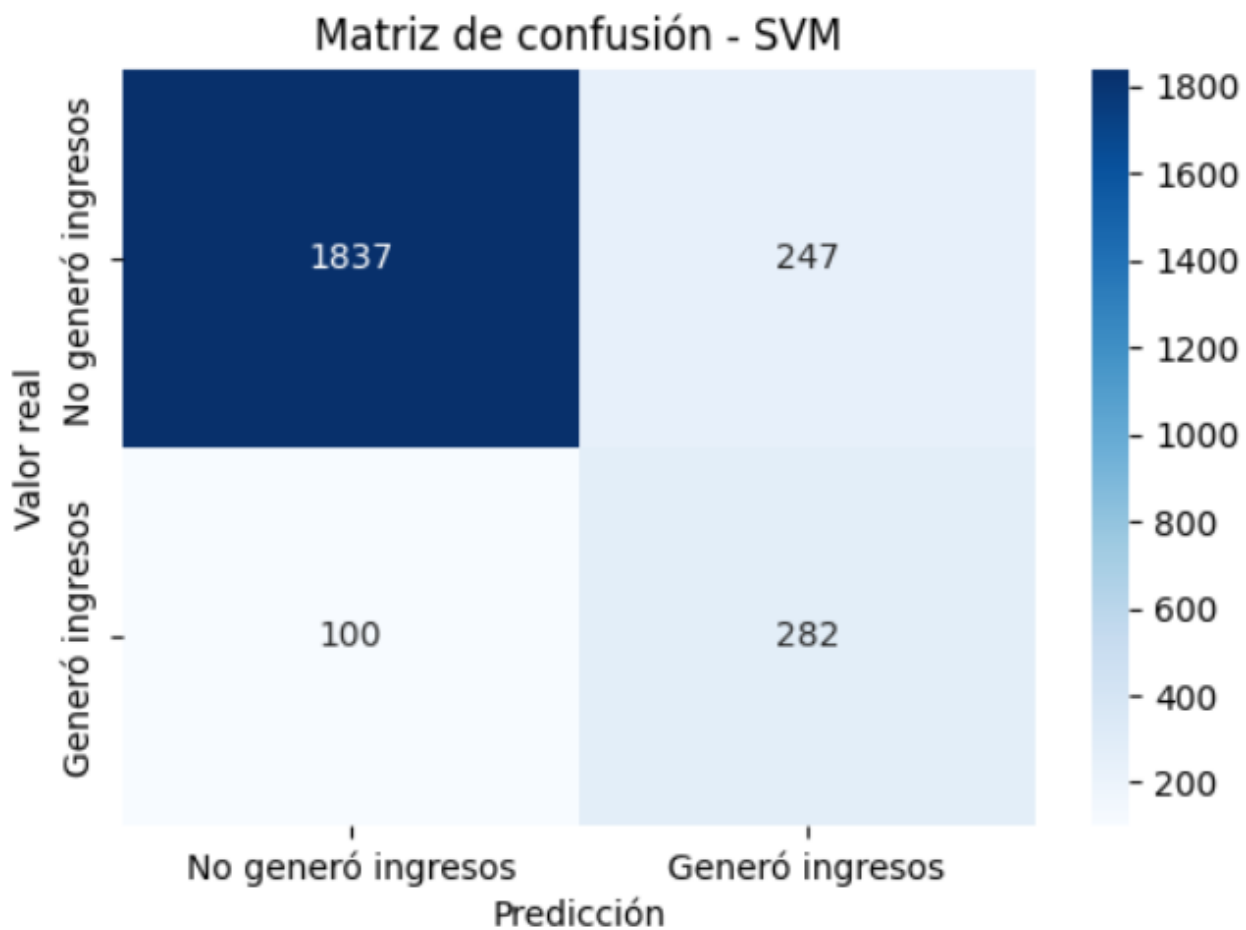
Accuracy SVM por fold: [0.85808414 0.87987836 0.85554992 0.87379625 0.87221095]
Accuracy promedio SVM: 0.8679039240702835
Accuracy Random Forest por fold: [0.89761784 0.89559047 0.89052205 0.90522048 0.90517241]
Accuracy promedio Random Forest: 0.8988246500166035
```

El accuracy obtenido mediante k-fold permite conocer el comportamiento promedio de cada modelo en diferentes particiones del conjunto de entrenamiento. Esto ayuda a evaluar si el modelo mantiene un rendimiento estable o si depende demasiado de una sola división de datos.

Matriz de confusión

La matriz de confusión permite observar los aciertos y errores de clasificación de cada modelo. En este proyecto, la clase 0 representa sesiones que no generaron ingresos, mientras que la clase 1 representa sesiones que sí generaron ingresos.

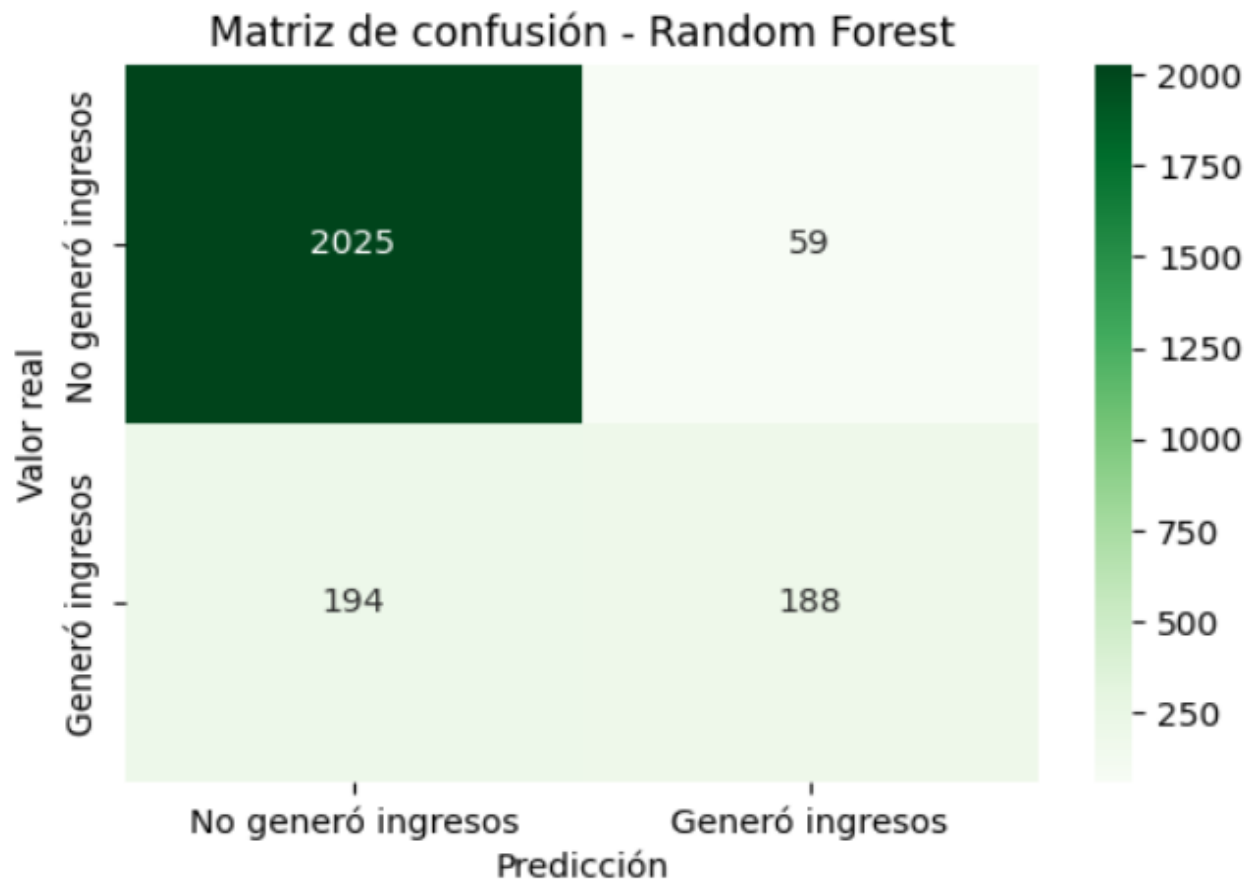
Figura 5.2. Matriz de confusión del modelo SVM



En la matriz de confusión de SVM se puede observar cuántas sesiones fueron clasificadas correctamente y cuántas fueron clasificadas de forma incorrecta. Esta visualización permite identificar si el modelo tiene mayor dificultad para detectar sesiones que generan ingresos o sesiones que no generan ingresos.

En la matriz de confusión de Random Forest se analiza el mismo comportamiento. Esta comparación permite observar cuál modelo comete menos errores y cuál identifica mejor la clase positiva, es decir, las sesiones que generaron ingresos.

Figura 5.3. Matriz de confusión del modelo Random Forest



Precision, recall y F1-score

Además del accuracy, se calcularon precision, recall y F1-score para cada modelo. La precision indica qué tan correctas fueron las predicciones positivas realizadas por el modelo. El recall indica qué proporción de casos positivos reales logró detectar el modelo. El F1-score

combina precision y recall en una sola métrica, por lo que resulta útil cuando existe desbalance entre clases.

Figura 5.4. Precision, recall y F1-score del modelo SVM

Reporte de clasificación - SVM				
	precision	recall	f1-score	support
0	0.95	0.88	0.91	2084
1	0.53	0.74	0.62	382
accuracy			0.86	2466
macro avg	0.74	0.81	0.77	2466
weighted avg	0.88	0.86	0.87	2466

El reporte de clasificación de SVM permite analizar el desempeño del modelo por cada clase. Esto es importante porque un modelo puede tener buen accuracy general, pero bajo desempeño al identificar la clase minoritaria.

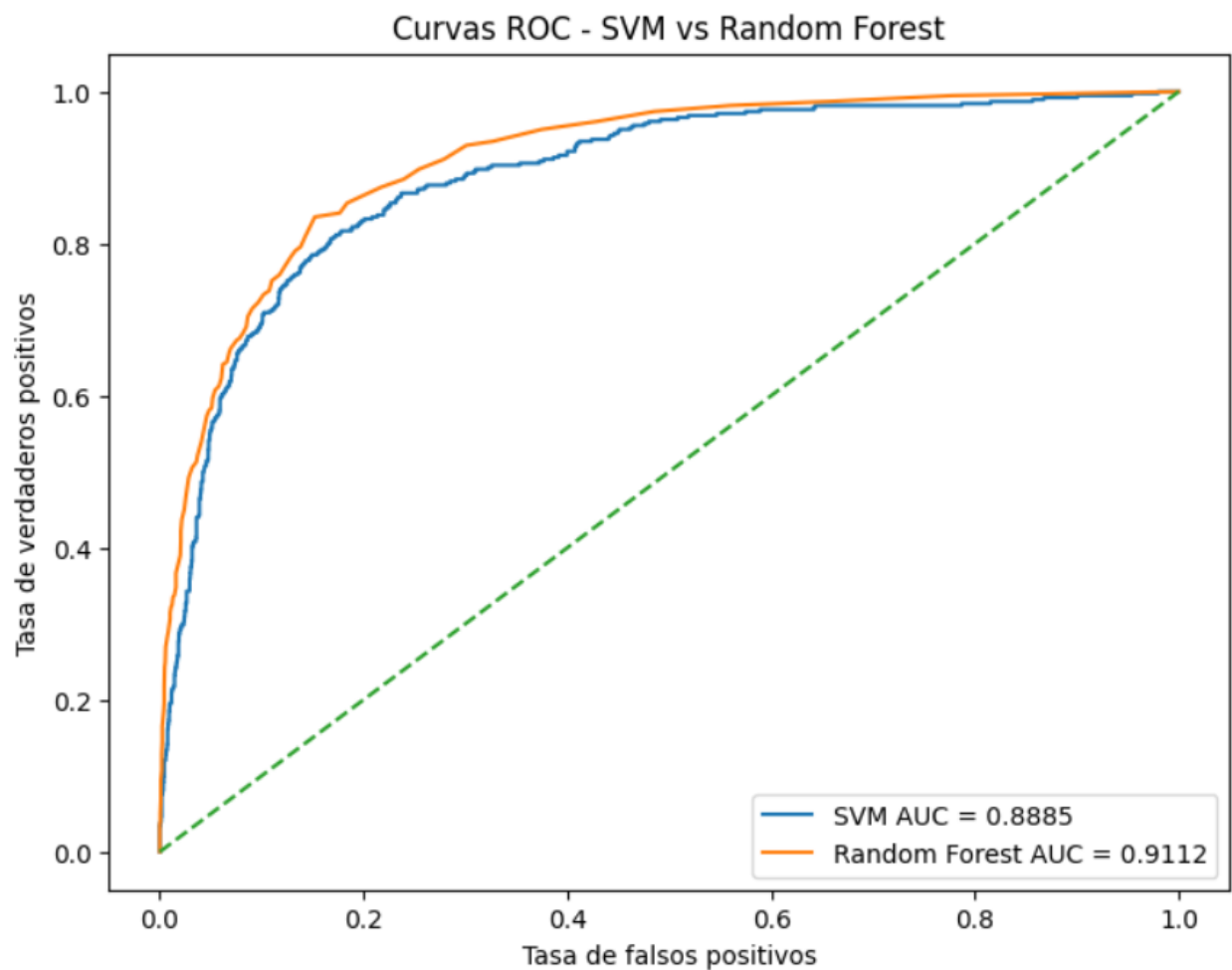
Figura 5.5. Precision, recall y F1-score del modelo Random Forest

Reporte de clasificación - Random Forest				
	precision	recall	f1-score	support
0	0.91	0.97	0.94	2084
1	0.76	0.49	0.60	382
accuracy			0.90	2466
macro avg	0.84	0.73	0.77	2466
weighted avg	0.89	0.90	0.89	2466

Curvas ROC y AUC

Como el problema de clasificación es binario, se generaron curvas ROC para ambos modelos. La curva ROC permite comparar la tasa de verdaderos positivos contra la tasa de falsos positivos. Además, se calculó el valor AUC, el cual resume el desempeño del modelo para distinguir entre las dos clases.

Figura 5.6. Curvas ROC y AUC de los modelos SVM y Random Forest

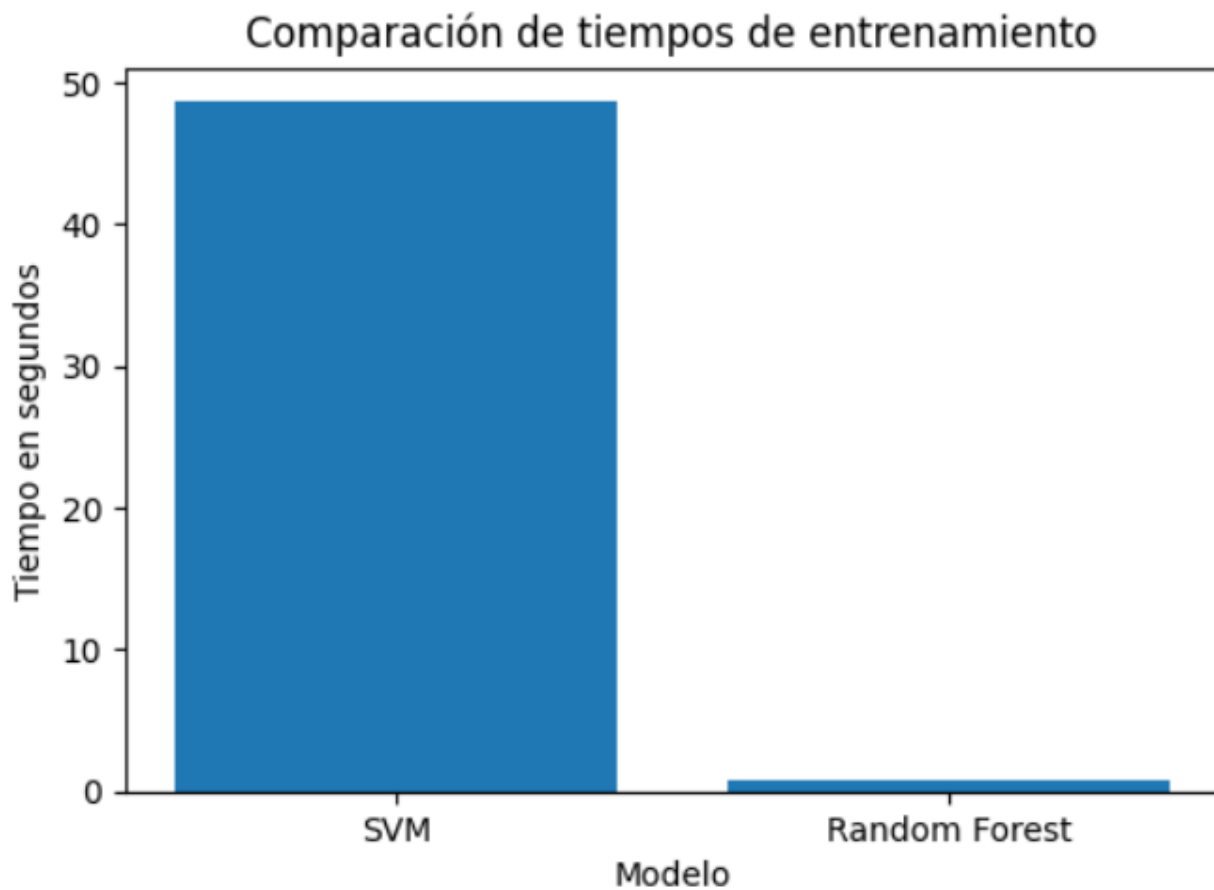


Un valor AUC más cercano a 1 indica que el modelo tiene mejor capacidad para distinguir entre sesiones que generan ingresos y sesiones que no generan ingresos. Por lo tanto, esta métrica permite complementar la comparación entre SVM y Random Forest.

Tiempo de entrenamiento

También se comparó el tiempo de entrenamiento de ambos modelos. El modelo SVM obtuvo un tiempo aproximado de **48.62 segundos**, mientras que Random Forest obtuvo un tiempo aproximado de **0.72 segundos**.

Figura 5.7. Comparación de tiempos de entrenamiento entre modelos



Estos resultados muestran que Random Forest fue considerablemente más rápido de entrenar que SVM. Esta diferencia puede deberse a que el modelo SVM, especialmente al utilizar el kernel rbf y el parámetro `probability=True`, requiere un mayor costo computacional para encontrar una frontera de separación adecuada entre las clases.

Por otro lado, Random Forest presentó un tiempo de entrenamiento menor, posiblemente porque puede aprovechar el procesamiento paralelo mediante el parámetro `n_jobs=-1` y porque su entrenamiento se basa en la construcción de varios árboles de decisión. Aunque el tiempo de entrenamiento no determina por sí solo cuál modelo es mejor, sí permite comparar la eficiencia computacional de cada algoritmo.

En este caso, Random Forest no solo obtuvo un mejor accuracy aproximado, sino que también fue más eficiente en tiempo de entrenamiento, lo cual puede hacerlo más adecuado para un entorno real donde se requiera entrenar o actualizar el modelo con mayor rapidez.

Comparación entre modelos

Al comparar los resultados obtenidos, se observa que Random Forest obtuvo un mejor desempeño general que SVM. El accuracy promedio de Random Forest fue de aproximadamente **0.89**, mientras que SVM obtuvo un accuracy aproximado de **0.86**. Esto indica que Random Forest logró clasificar correctamente una mayor proporción de sesiones.

En cuanto al tiempo de entrenamiento, también se observó una diferencia importante. SVM tardó aproximadamente 48.62 segundos, mientras que Random Forest tardó aproximadamente **0.71 segundos**. Por lo tanto, Random Forest no solo obtuvo un mejor accuracy, sino que también fue más rápido de entrenar.

Tabla 2

Modelo	Accuracy aproximado	Tiempo de entrenamiento
SVM	0.86	48.62 segundos
Random Forest	0.89	0.71 segundos

A partir de estos resultados, Random Forest puede considerarse el modelo con mejor desempeño en este proyecto. Una posible razón es que Random Forest combina varios árboles de decisión, lo que le permite capturar relaciones no lineales y patrones complejos dentro de los datos. Además, es menos sensible al escalado de variables y puede manejar mejor combinaciones de características generadas mediante One-Hot Encoding.

Por otro lado, SVM también logró un desempeño aceptable, pero presentó un menor accuracy y mayor tiempo de entrenamiento. Esto puede deberse a que SVM con kernel rbf requiere mayor costo computacional para encontrar una frontera de separación adecuada entre las clases.

En conclusión, para este dataset y bajo las condiciones utilizadas, Random Forest resultó ser el modelo más adecuado, ya que ofreció mejor rendimiento predictivo y menor tiempo de entrenamiento. Sin embargo, ambos modelos permitieron comparar dos enfoques diferentes de clasificación: SVM basado en separación de clases mediante un hiperplano y Random Forest basado en la combinación de múltiples árboles de decisión.

VI. Conclusión

A partir del desarrollo de este proyecto se logró aplicar la metodología CRISP-DM a un problema de clasificación utilizando el dataset **Online Shoppers Purchasing Intention Dataset**. Durante el proceso se aprendió que los datos de navegación de una tienda en línea pueden aportar información importante sobre la intención de compra de los usuarios. Variables como PageValues, ProductRelated, ProductRelated_Duration, BounceRates y ExitRates permitieron analizar patrones relacionados con el comportamiento del usuario dentro del sitio web.

También se comprendió que el proceso de clasificación no consiste únicamente en entrenar un modelo, sino que requiere varias etapas previas. Primero fue necesario entender el problema de negocio, conocer el dataset, realizar análisis exploratorio, tratar valores faltantes, convertir variables categóricas a numéricas, aplicar escalado y seleccionar correctamente las variables independientes y la variable objetivo. Esto permitió preparar los datos de manera adecuada antes de entrenar los modelos.

Una de las principales limitaciones del dataset es que representa sesiones de navegación, pero no incluye información más profunda sobre los usuarios, como historial de compras, preferencias personales, productos específicos consultados o razones por las que abandonaron el sitio. Además, la variable objetivo Revenue presenta un desbalance entre clases, ya que la mayoría de las sesiones no generaron ingresos. Esto puede afectar el desempeño de los modelos, especialmente al momento de detectar correctamente las sesiones que sí terminan en compra.

Respecto a los algoritmos utilizados, SVM presentó un desempeño aceptable, pero tuvo mayor tiempo de entrenamiento. En este proyecto obtuvo un accuracy aproximado de **0.86** y un tiempo de entrenamiento de **48.62 segundos**. Esto muestra que, aunque SVM puede ser útil para

clasificación, también puede ser más costoso computacionalmente, especialmente al utilizar kernel rbf y el parámetro probability=True.

Por otro lado, Random Forest obtuvo mejores resultados generales, con un accuracy aproximado de **0.89** y un tiempo de entrenamiento de **0.72 segundos**. Esto indica que fue más eficiente y obtuvo mejor desempeño en este caso. Sin embargo, Random Forest también tiene limitaciones, ya que puede ser menos interpretable que un árbol de decisión simple y puede requerir ajustes de hiperparámetros para mejorar aún más su rendimiento.

Este estudio podría mejorarse utilizando técnicas adicionales, como búsqueda de hiperparámetros con GridSearchCV o RandomizedSearchCV, balanceo de clases mediante técnicas como SMOTE, validación con más particiones o comparación con otros algoritmos. También podrían probarse modelos como k-NN, XGBoost, Árbol de Decisión, Regresión Logística o Naive Bayes. Además, sería útil contar con más información del contexto de compra, como categorías de productos, historial del cliente, promociones activas o comportamiento previo del usuario.

Aplicar este modelo en un entorno real también tendría riesgos. Por ejemplo, si el modelo clasifica incorrectamente a un usuario, la empresa podría tomar decisiones equivocadas, como no mostrar promociones a un cliente con alta intención de compra o invertir recursos en usuarios con baja probabilidad de comprar. También existe el riesgo de depender demasiado del modelo sin considerar otros factores comerciales, humanos o externos.

Las implicaciones prácticas de usar este modelo serían importantes para una tienda en línea. Sus resultados podrían apoyar decisiones relacionadas con marketing digital, segmentación de clientes, promociones personalizadas, recomendaciones de productos y mejora de la

experiencia del usuario. Sin embargo, el modelo debería utilizarse como una herramienta de apoyo y no como la única base para tomar decisiones.

En conclusión, el proyecto permitió comprender cómo aplicar un proceso completo de clasificación siguiendo CRISP-DM. Se identificó que Random Forest fue el modelo con mejor desempeño general para este dataset, debido a que obtuvo mayor accuracy y menor tiempo de entrenamiento en comparación con SVM. Aun así, ambos modelos permitieron analizar el problema desde enfoques diferentes y demostrar la importancia de preparar correctamente los datos antes de evaluar el desempeño de los algoritmos.

VII. Bibliografia

Scikit-learn. (s. f.). Support Vector Machines. <https://scikit-learn.org/stable/modules/svm.html> Scikit-learn. (s. f.). RandomForestClassifier. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

Scikit-learn. (s. f.). StandardScaler. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

Sakar, C. O., & Kastro, Y. (2018). Online Shoppers Purchasing Intention Dataset. UCI Machine Learning Repository. <https://doi.org/10.24432/C5F88Q>